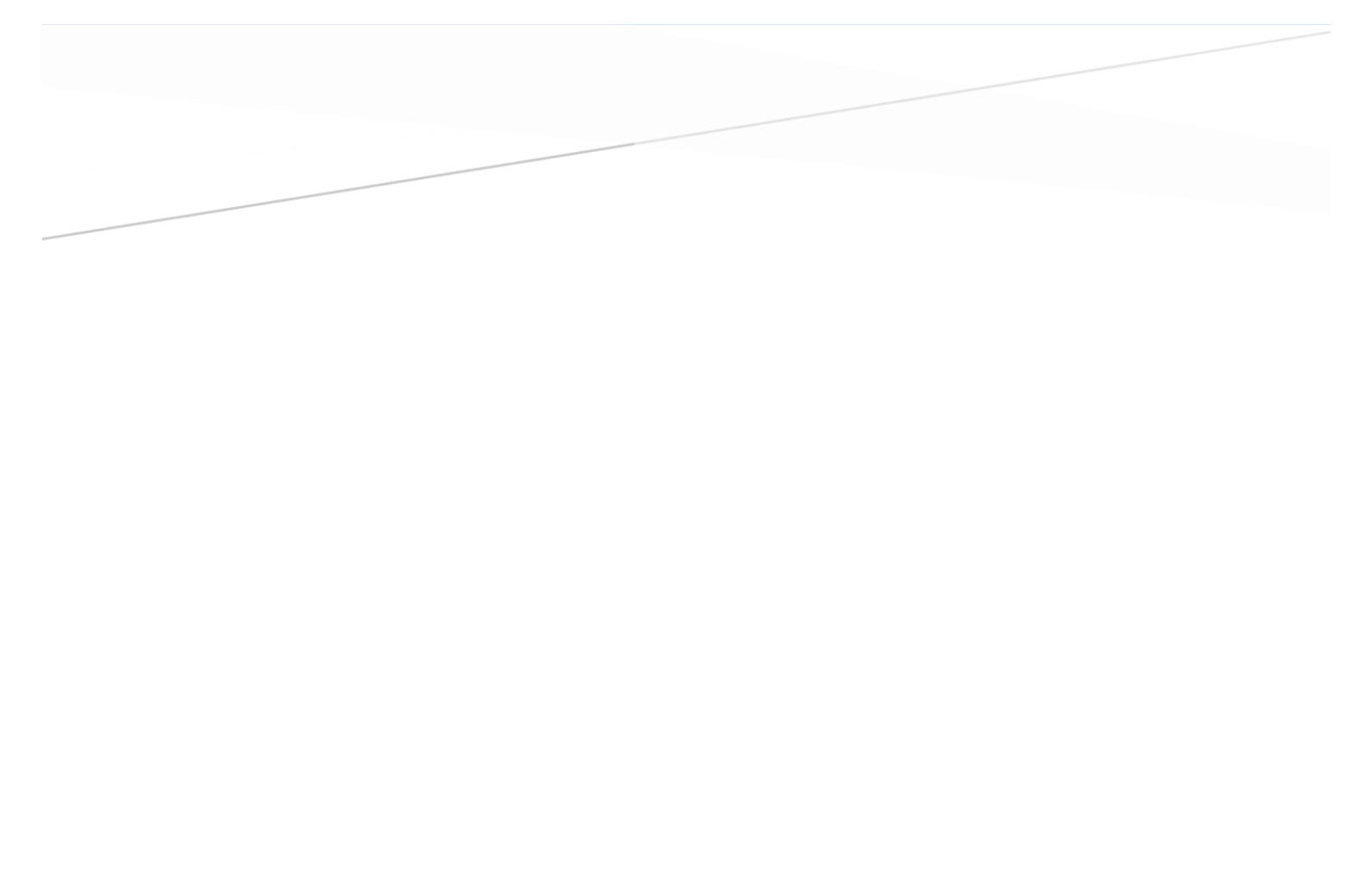




RESUMEN JAVA

`PROGRAMACIÓN

Contenido

1	1ª EVALUACIÓN.....	4
2	MI PRIMER PROGRAMA	4
3	MAIN	4
4	MOSTRAR TEXTO EN PANTALLA.	5
5	FORMATEO DE TEXTO.....	5
5.1	Printf.....	5
5.2	FORMATEO USANDO CLASES.....	6
6	VARIABLES.....	7
7	CONSTANTES.....	9
8	AMBITO DE LAS VARIABLES.	9
9	OPERADORES ARITMÉTICOS.....	9
10	ASIGNACIÓN DE VALORES A VARIABLES.....	10
11	PARSEAR O CASTING.....	11
12	WRAPERS o clases envoltorios.....	12
13	LECTURA DE DATOS DESDE TECLADO.	13
13.1	la clase Scanner	14
13.2	Console.readPassword()	14
14	LA CLASE STRING.	15
14.1	Métodos de la clase <i>String</i>	15
14.2	ValueOf	18
14.3	Métodos Java String toLowerCase () y toUpperCase	18
14.4	Averiguar el tipo de variables o clases que estamos utilizando: Instanceof, getClass	19
15	SENTENCIA CONDICIONAL (IF Y SWITCH).....	20
16	BUCLES.....	25
17	PAQUETES EN JAVA.	27
18	LIBRERIAS EN JAVA.	28
18.1	Clase Math.....	28
19	GENERACIÓN DE NÚMEROS ALEATORIOS	30
20	2ª EVALUACIÓN	32
21	ARRAY Y CLASE ARRAY.....	32
21.1	La clase Array.....	35
22	MÉTODOS O FUNCIONES.....	43
22.1	Uso de métodos void.....	52
23	PROGRAMACIÓN ORIENTADO A OBJETOS (POO)	53

23.1	Encapsulamiento y ocultación.....	54
23.2	Métodos	54
23.3	Método constructor	56
23.4	Método Getter/Setter	57
23.5	Enum.....	59
23.6	Clases abstractas.....	60
23.7	Sobrecarga de constructores.	64
23.8	Atributos y métodos de clase.....	64
23.9	Interfaces	68
23.10	Diferencias entre clases abstractas e interfaces	71
23.11	Arrays de objetos.	78
24	COLECCIONES Y MÉTODOS.....	83
24.1	Interfaz Collection	85
24.2	Interfaces set, map, list.....	85
24.3	Interfaz Set: HashSet, LinkedHashSet, TreeSet	87
24.4	Interfaz list, ArrayList, Stack(pila), LinkedList	89
24.5	Interfaz Queue(cola): PriorityQueue, ArrayQueue	96
24.6	Interfaz Map: HashMap, LinkedHashMap, TreeMap	100
25	EXCEPCIONES	105
25.1	Control de varios tipos de excepciones	106
25.2	Lanzamiento de excepciones con throw.....	107
25.3	Declaración de un método con throws	107
25.4	Creación de Excepciones propias.	108
25.5	Recomendaciones sobre el tratamiento de excepciones.	110
26	POLIMORFISMO	110
27	APENDICE	112
27.1	Expresiones lambda.....	112
27.2	JOptionPane.	115
27.2.1	showMessageDialog.....	116
27.2.2	Método JOptionPane.showInputDialog()	117
27.2.3	Método JOptionPane.showConfirmDialog()	119
27.2.4	Método JOptionPane.showOptionDialog().....	120

1 1ª EVALUACIÓN

2 MI PRIMER PROGRAMA

```
HolaMundo.java x
1 /**
2  * Muestra por pantalla la frase "¡Hola mundo!"
3  *
4  * @author Luis J. Sánchez
5  */
6 public class HolaMundo { // Clase principal
7     public static void main(String[] args) {
8         System.out.println("¡Hola mundo!");
9     }
10 }
```

Jav

1º el nombre de la clase preferentemente escribir primera letra en mayúscula. Si se compone de dos nombre cada nombre en mayúscula y juntas.

2º El nombre del fichero igual que el nombre de la clase, en nuestro ejemplo

HolaMundo.java

Saber más

Convencionalismos

- Los nombres de las clases deben ser sustantivos, en mayúsculas y minúsculas, con la primera letra de cada palabra interna en mayúscula
- Los métodos deben ser verbos, en mayúsculas y minúsculas, con la primera letra de cada palabra interna (a partir de la segunda) en mayúscula.
- No debería comenzar con un guión bajo ('_') o caracteres, como por ejemplo, un signo de dólar '\$'. Debe ser mnemotécnico, es decir, diseñado para indicar al observador casual la intención de su uso. Se deben evitar los nombres de variable de un carácter, excepto para variables temporales. Los nombres comunes para las variables temporales son: i, j, k, m y n para enteros; c, d y e para los caracteres.
- Debería estar todo en mayúsculas con palabras separadas por guiones bajos ("_").

3 MAIN

Es un método que nos permite la ejecución de nuestro programa, tiene que estar obligatoriamente.

- **Public**

Ha de ser siempre public, es decir publico

- **Static**

Podemos acceder a métodos sin necesidad de instanciar la clase.

- Void

La palabra void indica que el método main no retorna ningún valor.

Nota

Los **comentarios** se ponen

// una línea en pantalla

/* finaliza con */ varias líneas

/** y finaliza con */ para la documentación de javadoc

4 MOSTRAR TEXTO EN PANTALLA.

Existen varios métodos para mostrar texto en la consola:

System.out.print(). No añade un salto de línea

System.out.println(). Si añade un salto de línea

Nota

Los saltos de línea se pueden realizar con \n

Ejemplo:

```
System.out.print("hola\n");
```

```
System.out.print("adios\n");
```

5 FORMATEO DE TEXTO.

5.1 Printf

Permite formatear la salida que se pretende mostrar por pantalla

Ejemplo:

```
public class SalidaFormateada02 {  
    public static void main(String[] args) {  
        System.out.println(" Artículo Precio/caja N° cajas");  
        System.out.println("-----");  
        System.out.printf("%-10s %8.2f %6d\n", "manzanas", 4.5, 10);  
        System.out.printf("%-10s %8.2f %6d\n", "peras", 2.75, 120);  
        System.out.printf("%-10s %8.2f %6d\n", "aguacates", 10.0, 6);  
    }  
}
```



INDICADORES DE FORMATO			
Indicador	Significado	Indicador	Significado
-	Alineación a la izquierda	+	Mostrar signo + en números positivos
(	Los números negativos se muestran entre paréntesis	0	Rellenar con ceros
,	Muestra el separador decimal		

CARACTERES DE CONVERSIÓN			
Carácter	Tipo	Carácter	Tipo
d	Número entero en base decimal	X, x	Número entero en base hexadecimal
f	Número real con punto fijo	s	String
E, e	Número real notación científica	S	String en mayúsculas
g	Número real. Se representará con notación científica si el número es muy grande o muy pequeño	C, c	Carácter Unicode. C: en mayúsculas

5.2 FORMATEO USANDO CLASES

Otra manera más habitual es usar clases para formatear textos, por ejemplo, queremos formatear un número a decimal en pantalla, como podemos ver en el ejemplo:

Usaremos la clase DecimalFormat, al instanciar la clase, creando un objeto, podemos utilizar el método format del objeto creado.

Ejemplo

```
import java.util.Scanner;
import java.text.DecimalFormat;
public class Ej7_ED {

    public static void main(String[] args) {

        Scanner c=new Scanner(System.in);
        DecimalFormat formateador = new DecimalFormat("####.##");
        System.out.print("Por favor, introduzca la base imponible (precio del artículo sin IVA): ");
```

```

double baseImponible = Double.parseDouble(System.console().readLine());
    double total, iva=0;
    iva=baseImponible * 0.21;
    total=baseImponible+iva;
    System.out.println ("indica el iva " + formateador.format (iva));
    System.out.printf("-----\n");
    System.out.println ("el total es " + formateador.format (total));
}
}

```

6 VARIABLES.

Son espacios de la memoria reservados de una manera “variable”, es decir sus valores pueden cambiar y si no se realiza ninguna acción para guardar dichos valores, su contenido desaparece.

Tipos

Enteros (int y long)

Generalmente usamos int, long es para valores numéricos muy grandes.

Ejemplo con enteros y decimales

```

public class VariablesConDecimales {
public static void main(String[] args) {
int x; // Se declaran las variables x e y
double y; // de tal forma que puedan almacenar decimales.
x = 7;
y = 25.01;
System.out.println(" x vale " + x);
System.out.println(" y vale " + y);
}
}

```

Números decimales (double y float)

Double son más precisas que float.

Cadenas de caracteres (String)

Ejemplo

```

public class UsoDeStrings {
public static void main(String[] args) {
String miPalabra = "cerveza";
String miFrase = "¿dónde está mi cerveza?";
System.out.println("Una palabra que uso con frecuencia: " + miPalabra);
System.out.println("Una frase que uso a veces: " + miFrase);
}
}

```

Caracteres (char)

Un carácter suelto como una letra o un signo de puntuación se puede almacenar en una variable de tipo char. El carácter debe ir entrecomillado utilizando las comillas simples (').

Ejemplo:

```
public class UsoDeChar {
public static void main(String[] args) {
char letra1 = 'c';
char letra2 = 'a';
char letra3 = 's';
char letra4 = 'a';
System.out.println("letra1: " + letra1);
System.out.println("letra3: " + letra3);
System.out.println("todas las letras juntas: " + letra1 + letra2 + letra3 + letra4);
}
}
```

Tipos en tablas

Tipos de datos primitivos

Nombre	Tipo de dato	Ocupa	Rango	Valor defecto
byte	Entero (signo)	1 byte	-128 a 127	0
short	Entero (signo)	2 bytes	-32768 a 32767	0
int	Entero (signo)	4 bytes	-2^{31} a $2^{31}-1$	0
	Entero		0 a $2^{32}-1$	
long	Entero (signo)	8 bytes	-2^{63} a $2^{63}-1$	0L
	Entero		0 a $2^{64}-1$	
float	Decimal simple	4 bytes	Punto flotante 32-bit IEEE 754	0.0f
double	Decimal doble	8 bytes	Punto flotante 64-bit IEEE 754	0.0d
char	Carácter simple	2 bytes	'\u0000' a '\uffff' (65,535)	'\u0000'
boolean	Booleano	1 byte	true o false	false

Conversiones seguras, se definen porque el origen es menor que el destino, por tanto no se va a perder información:

Tipo de dato origen	Tipo de dato destino
byte	double, float, long, int, char, short
char	double, float, long, int
short	
int	double, float, long
long	double, float
float	double

7 CONSTANTES.

Una constante es un elemento que mantiene un valor inalterable a lo largo de toda la vida del programa, se antepone al nombre de la variable `static final`:

`static final int i = 21;`

Nota

Suelen definirse antes del método `main`

8 AMBITO DE LAS VARIABLES.

MODIFICADOR	CLASE	PACKAGE	SUBCLASE	TODOS
public	Sí	Sí	Sí	Sí
protected	Sí	Sí	Sí	No
No especificado	Sí	Sí	No	No
private	Sí	No	No	No

9 OPERADORES ARITMÉTICOS.

emáticas. Los operadores aritméticos de Java son los siguientes:

OPERADOR	NOMBRE	EJEMPLO	DESCRIPCIÓN
+	suma	<code>20 + x</code>	suma dos números
-	resta	<code>a - b</code>	resta dos números
*	multiplicación	<code>10 * 7</code>	multiplica dos números
/	división	<code>altura / 2</code>	divide dos números
%	resto (módulo)	<code>5 % 2</code>	resto de la división entera
++	incremento	<code>a++</code>	incrementa en 1 el valor de la variable
--	decremento	<code>a--</code>	decrementa en 1 el valor de la variable

Ejemplo

```
public class UsoDeOperadoresAritmeticos {  
    public static void main(String[] args) {  
        int x;  
        x = 100;  
        System.out.println(x + " " + (x + 5) + " " + (x - 5));  
        System.out.println((x * 5) + " " + (x / 5) + " " + (x % 5));  
    }  
}
```

Precedencia

Es un orden de evaluación, es decir el que se ejecutará primero:

Precedencia
expr++ expr--
++expr --expr +expr -expr ~ !
* / %
+ -
<< >> >>>
< > <= >= instanceof
== !=
&
^
&&
= += -= *= /= %= &= ^= = <<= >>= >>>=

10 ASIGNACIÓN DE VALORES A VARIABLES.

Se usa para dar valor a las variables

Ejemplo

```
public class Asignaciones {  
    public static void main(String[] args) {  
        int x = 2;  
        int y = 9;  
        int sum = x + y;  
    }  
}
```

```
System.out.println("La suma de mis variables es " + sum);
int mul = x * y;
System.out.println("La multiplicación de mis variables es " + mul);
}
}
```

11 PARSEAR O CASTING.

En ocasiones es necesario convertir una variable (o una expresión en general) de un tipo a otro. Simplemente hay que escribir entre paréntesis el tipo que se quiere obtener. Experimenta con el siguiente programa y observa los diferentes resultados, en este caso también podemos denominarlo conversiones implícitas.

que se obtienen.

```
public class ConversionDeTipos {
    public static void main(String[] args) {
        int x = 2;
        int y = 9;
        double division;
        division = (double) y / (double) x;
        // Descomenta la siguiente línea y observa cómo cambia el resultado.
        // division = y / x;
        System.out.println("El resultado de la división es " + division);
    }
}
```

Integer.parseInt

```
Integer numero = Integer.parseInt("12");
```

Conversiones implícitas

Se realizan de forma automática, como vemos en el ejemplo siguiente:

```
byte b = 1;
short s = b;
int i = s;
long l = i;
float f = l;
double d = f;
```

Conversiones implícitas

Es más segura que el ejemplo anterior, veamos un ejemplo

```
int i = 127;
byte b = (byte) i;
```

12 WRAPERS o clases envoltorios

Creamos objetos que contengan un dato de tipo primitivo.

CUADRO 4.4
Correspondencia tipo primitivo – Wrapper Class

Primitivo	boolean	byte	char	double	float	int	long	short
Wrapper	Boolean	Byte	Character	Double	Float	Integer	Long	Short

CUADRO 4.5
Métodos de cualquier Wrapper Class

toString()	Retorna un String cuyo literal es el valor del dato primitivo.
x.compareTo(y)	Retorna 0 si los datos contenidos en x e y son iguales. Retorna 1 si el valor de x es mayor al de y. Retorna -1 si el valor de x es menor al de y.
x.equals(y)	Retorna <i>true</i> si los datos contenidos en x e y son iguales y <i>false</i> en caso contrario.

Saber más

Autoboxing y unboxing

Dado que existe correspondencia directa entre tipos primitivos y clases envoltorio, a partir del *JDK 5* se incluyó la conversión automática entre ambos tipos:

- Si la conversión es de tipo primitivo a envoltorio, se denomina *autoboxing*.
- Si la conversión es de tipo envoltorio a primitivo, se denomina *unboxing*.

A continuación, puede verse un ejemplo *autoboxing* y *unboxing*:

```
Integer a = 11, b = 20; // autoboxing
int c = b - a;          // unboxing
```

13 LECTURA DE DATOS DESDE TECLADO.

Para recoger datos por teclado usamos

`System.console().readLine()`.

Nota debemos asignarlo a una variable como se ve en la imagen inferior:

```
String nombre;
nombre = System.console().readLine();
```

Ejemplo

```
public class DimeTuNombre {
    public static void main(String[] args) {
        String nombre;
        System.out.print("Por favor, dime cómo te llamas: ");
        nombre = System.console().readLine();
        System.out.println("Hola " + nombre + ", ¡encantado de conocerte!");
    }
}
```

Ejemplo

```
public class LeeNotas {
    public static void main(String[] args) {
        String linea, asignatura;
        linea = System.console().readLine("Por favor, introduce una nota ");
        int nota = Integer.parseInt( linea );
        asignatura = System.console().readLine("di la asignatura ");
        System.out.print(nota+" "+asignatura);
    }
}
```

Usando números

Por defecto la instrucción `readline` es `string`, si queremos usar números debemos convertir ese `readline` en número, de esta manera:

Ejemplo:

```
public class LeeNumeros {
```

```

public static void main(String[] args) {
    String linea;
    System.out.print("Por favor, introduce un número: ");
    linea = System.console().readLine();
    int primerNumero;
    primerNumero = Integer.parseInt( linea );
    System.out.print("introduce otro, por favor: ");
    linea = System.console().readLine();
    int segundoNumero;
    segundoNumero = Integer.parseInt( linea );
    int total;
    total = (2 * primerNumero) + segundoNumero;
    System.out.print("El primer número introducido es " + primerNumero);
    System.out.println(" y el segundo es " + segundoNumero);
    System.out.print("El doble del primer número más el segundo es ");
    System.out.print(total);
}
}

```

13.1 la clase Scanner

Otro método es usar. Es el más indicado en IDE como Eclipse o NetBeans

Para ello debemos crear un objeto de esta clase, esto nos va a permitir mayor potencia, porque podremos utilizar los métodos de esa clase:

Ejemplo:

```

import util.java.Scanner
public class LeeDatosScanner01 {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("Introduce tu nombre: ");
        String nombre = s.nextLine();
        System.out.print("Introduce tu edad: ");
        int edad = Integer.parseInt(s.nextLine());
        System.out.println("Tu nombre es " + nombre + " y tu edad es " + edad);
    }
}

```

Depende del tipo de variable:

```
nextBoolean()
```

```
nextDouble()
```

13.2 Console.readPassword()

Lee una contraseña de la consola sin hacer eco de la misma. Es decir, mientras el usuario la escribe no se ven los caracteres que está introduciendo. En este ejemplo podemos poner el texto que acompaña a los campos dentro de readLine y de readPassword.

Ejemplo

```
import java.io.*;
public class Password {

    public static void main (String[] args) {
        Console c=System.console();
        if (c==null){
            System.err.println("error");
        }else {

            String nombre=c.readLine(" di tu nombre ");
            String password=String.valueOf(c.readPassword("tu password "));
            System.out.println(" tu nombre es "+nombre +" tu contraseña es "+password);

        }

    }

}
```

En el caso de usar JOptionPane, tendríamos que usar este ejemplo:

En este caso debemos crear un objeto basado en la clase JPasswordField

```
import java.io.*;
import javax.swing.*;
public class Password2 {

    public static void main (String[] args) {

        JPasswordField pwd = new JPasswordField(10);
        int action = JOptionPane.showConfirmDialog(null, pwd,"Enter password",
        JOptionPane.OK_CANCEL_OPTION);
        String contra;
        contra=new String(pwd.getPassword());

        if(action < 0 || contra.equals("")){ JOptionPane.showMessageDialog(null,"Cancel,X or escape
        key selected");}

        else { JOptionPane.showMessageDialog(null, "your password is "+contra);
        System.exit(0);}

    }

}
```

14 LA CLASE STRING.

los **String** en Java no son del tipo de dato simple, sino instancias de la clase string, son objetos, como todo lo que compone a este lenguaje y tienen una sintaxis especial para representar cadenas de caracteres. es un envoltorio para los datos de tipo primitivo de cadenas y proporciona métodos que permiten manipular esas cadenas.

14.1 Métodos de la clase *String*

Los métodos predeterminados que posee **String** nos dan la posibilidad de realizar muchas cosas, aquí presentamos algunos de ellos:

MÉTODO	DESCRIPCIÓN
length()	Nos sirve para conocer la longitud de la cadena
indexOf('character')	Nos entrega la ubicación, el índice, de la primera aparición de
lastIndexOf('character')	Nos entrega la ubicación, el índice, de la última aparición del
charAt(n)	Nos muestra qué carácter se encuentra en la posición solicitada
substring(n1,n2)	Este método nos devuelve la subcadena que se encuentra entre
toUpperCase()	Convierte toda la cadena a mayúsculas
toLowerCase()	Convierte toda la cadena a minúsculas.
equals(«cad»)	Compara dos cadenas y nos informa True si son iguales
equalsIgnoreCase(«cad»)	Compara dos cadenas y nos informa True si son iguales y no c
compareTo(OtroString)	Uno de los métodos más utilizados, nos devuelve 0 si ambas son iguales, <0 si la primera es alfabéticamente menor que la segunda ó >0 si la primera es a
String trim()	Devuelve la copia de la cadena, elimina los espacios en blanco al principio y al final, así como los espacios en blanco intermedios.
valueOf(N)	Nos sirve para convertir el valor N a String. N puede ser de cualquier tipo
String replace (char oldChar, char newChar)	Devuelve una nueva cadena al reemplazar un carácter por otro

Ejemplos

código	resultado
"Miguel".length()	6
"Miguel".equals("Miguel")	true
"Miguel".equals("miguel")	false
"Miguel".equalsIgnoreCase("miguel")	true
"Miguel".compareTo("Saturnino")	-6 (cualquier entero < 0)
"Miguel".compareTo("Miguel")	0
"Miguel".compareTo("Michelin")	4 (cualquier entero > 0)
"Miguel".charAt(1)	'i'
"Miguel".charAt(4)	'e'
"Miguel".toCharArray()	{ 'M', 'i', 'g', 'u', 'e', 'l' }
"Miguel".substring(1, 4)	"igu"
"Miguel".substring(1)	"iguel"
"tragaldabas".indexOf('a')	2
"tragaldabas".lastIndexOf('a')	9
"tragaldabas".startsWith("tragón")	false
"tragaldabas".endsWith("dabas")	true
"tragaldabas".split("a")	{ "tr", "g", "ld", "b", "s" }

Ejemplo de Split


```
String nombre="tra/gal/dab/as";

    String caracter[]=nombre.split("/");
    for (int i=0;i<caracter.length;i++){
        System.out.print(caracter[i]);
    }
}
```

Ejemplo de uso de String, JOptionPane, bucles y condicionales.

```
//Pedir correo y comprobar si esta "@" y "."
//
import java.io.*;
import javax.swing.*;

public class ComprobacionMail {
    public static void main (String [] args) {

        for (int i = 3; i >-1; i--) {

            //////////////////////////////////////////////////usuario,correo////////////////////////////////////
            String usuario = JOptionPane.showInputDialog(null, "Usuario: ",
"Registro", 1);
            String correo = JOptionPane.showInputDialog(null, "introduzca su
correo electronico", "Correo", 1);

            //////////////////////////////////////////////////sacando si hay @ o no////////////////////////////////
            int confirmacion = correo.indexOf("@");
            int confirmacion1 = correo.indexOf(".");
            if ((confirmacion != -1) & (confirmacion1 != -1)){

            ////////////////////////////////////////////////// contraseña////////////////////////////////
                JPasswordField pwd = new JPasswordField(10);

                int action = JOptionPane.showConfirmDialog(null, pwd,"Enter
password",JOptionPane.OK_CANCEL_OPTION);

                String contra=new String(pwd.getPassword());
            //////////////////////////////////////////////////condicionales////////////////////////////////

                //if ((confirmacion != -1) & (confirmacion1 != -1)){
                if (!contra.equals("")){
                    JOptionPane.showMessageDialog(null, "Correcto, tu usuario es
"+usuario+ " tu correo es:"+correo+" y tu contraseña es;
"+contra,"Resumen",1);
                    //System.out.println("Correcto, tu usuario es "+usuario+ " tu
correo es:"+correo+" y tu contraseña es; "+contra);
                }
            }
        }
    }
}
```

```

        break;

    }

    else
        //{System.out.println("la contraseña no puede ir en blanco., tienes:
"+i+" intentos");}
        {JOptionPane.showMessageDialog(null, "la contraseña no puede ir
en blanco, intentos restantes"+i,"mensaje",0);}

    }
    else
        {JOptionPane.showMessageDialog(null, "el correo electronico esta mal
introducido, intentos restantes: "+i,"mensaje",0);}

}

}

}

```

14.2 ValueOf

El método valueOf es un método sobrecargado aplicable a numerosas clases de Java y que permite realizar conversiones de tipos.

```

public class Test{
public static void main(String args[]){
    Integer x =Integer.valueOf(9);
    Double c = Double.valueOf(5);
    Float a = Float.valueOf("80");
    System.out.println(x);
    System.out.println(c);
    System.out.println(a);

    }
}

```

14.3 Métodos Java String toLowerCase () y toUpperCase

Este método convierte en minúscula o mayúscula.

- toUpperCase; convierte a mayúsculas.
- toLowerCase: convierte a minúsculas.

Ejemplo

```
String sCadena = "Esto Es Una Cadena";
System.out.println(sCadena.toUpperCase()); //ESTO ES UNA CADENA
```

14.4 Averiguar el tipo de variables o clases que estamos utilizando: Instanceof, getClass .

El operador *instanceof* se utiliza para conocer si una variable es de un tipo determinado. El primer operador es la variable, mientras que el segundo es el tipo.

El valor obtenido al aplicar este operador será un valor booleano *true* en caso de que la variable sea del tipo indicado o *false* en caso contrario.

Ejemplo

Solo se puede realizar con clases y arrays:

```
public class EjemploInstanceOf {

    public static void main (String[] args) {

        String cad = "Hello world !";
        System.out.println(cad instanceof String);

    }

}
```

El resultado es true

GetClass

Realiza lo mismo que instanceof pero podemos usarlo con las variables, veamos un ejemplo:

```
public class EjemploInstanceOf {

    public static void main (String[] args) {

        int myInteger = 10;
        String myString = "Hola";
        double myDouble = 0;
        System.out.println("myInteger es de tipo " +
        ((Object)myInteger).getClass().getSimpleName());
        System.out.println("myString es de tipo " +
        ((Object)myString).getClass().getSimpleName());
        System.out.println("myDouble es de tipo " +
        ((Object)myDouble).getClass().getSimpleName());

    }

}
```

```

    }
}

```

- **Object**

Con este método, hacemos referencia a la variable que estamos usando: la tratamos como si fuera un objeto.

- **getClass**

El método `getClass()` se encuentra definido en `Object` como un método final, dicho método devuelve una representación en tiempo de ejecución de la clase del objeto sobre el cual podemos acceder a una serie de características del objeto por ejemplo: el nombre de la clase, el nombre de su superclase y los nombres de los interfaces que implementa

- **getSimpleName();**

Con este método hacemos referencia al tipo de datos de esa clase

15 SENTENCIA CONDICIONAL (IF Y SWITCH).

Sentencia if

Se utiliza la instrucción `if` y `else`, veamos un ejemplo:

```

public class SentenciaIf02 {
    public static void main(String[] args) {
        System.out.print("Por favor, introduce un número entero: ");
        String linea = System.console().readLine();
        int x = Integer.parseInt( linea );
        if (x < 0) {
            System.out.println("El número introducido es negativo.");
        } else {
            System.out.println("El número introducido es positivo.");
        }
    }
}

```

Operadores de comparación

Tabla 4.2.1. Operadores de comparación.

OPERADOR	NOMBRE	EJEMPLO	DESCRIPCIÓN
<code>==</code>	igual	<code>a == b</code>	a es igual a b
<code>!=</code>	distinto	<code>a != b</code>	a es distinto de b
<code><</code>	menor que	<code>a < b</code>	a es menor que b
<code>></code>	mayor que	<code>a > b</code>	a es mayor que b
<code><=</code>	menor o igual que	<code>a <= b</code>	a es menor o igual que b
<code>>=</code>	mayor o igual que	<code>a >= b</code>	a es mayor o igual que b

Operadores lógicos

lógicos de java:

Tabla 4.3.1. Operadores lógicos.

OPERADOR	NOMBRE	EJEMPLO	DEVUELVE VERDADERO CUANDO...
&&	y	(7 > 2) && (2 < 4)	las dos condiciones son verdaderas
	o	(7 > 2) (2 < 4)	al menos una de las condiciones es verdadera
!	no	!(7 > 2)	la condición es falsa

Elseif

Se usa para distintas condiciones unas dentro de otras

Ejemplo:

```
public class LeeNotas {
    public static void main(String[] args) {
        String linea, notaTexto;
        notaTexto="";
        System.out.print("Por favor, introduce una nota ");
        linea = System.console().readLine();
        int nota;
        nota = Integer.parseInt( linea );
        if (nota>=0 && nota<5 ){
            notaTexto="suspense";
        } else if(nota==5){
            notaTexto="suficiente";
        }
        else if(nota==6){
            notaTexto="bien";
        }
        else if(nota==7 || nota==8){
            notaTexto="notable";
        }
        else if(nota==9 || nota==10){
            notaTexto="suficiente";
        }
        else if (nota<0 || nota>10)
        { notaTexto="errora";
        }

        System.out.println("su nota es "+notaTexto);
    }
}
```

Sentencia switch (selección múltiple

Syntaxis:

```
switch(variable) {
```

```
case valor1:
sentencias
break;
case valor2:
sentencias
break;
.
.
.
default:
sentencias
}
```

Ejemplo:

```
public class SentenciaSwitch {
public static void main(String[] args) {
System.out.print("Por favor, introduzca un numero de mes: ");
int mes = Integer.parseInt(System.console().readLine());
String nombreDelMes;
switch (mes) {
case 1:
Sentencia condicional (if y switch) 39
nombreDelMes = "enero";
break;
case 2:
nombreDelMes = "febrero";
break;
case 3:
nombreDelMes = "marzo";
break;
case 4:
nombreDelMes = "abril";
break;
case 5:
nombreDelMes = "mayo";
break;
case 6:
nombreDelMes = "junio";
break;
case 7:
nombreDelMes = "julio";
break;
case 8:
nombreDelMes = "agosto";
break;
case 9:
nombreDelMes = "septiembre";
break;
case 10:
nombreDelMes = "octubre";
break;
case 11:
nombreDelMes = "noviembre";
break;
case 12:
nombreDelMes = "diciembre";
```

```

break;
default:
nombreDelMes = "no existe ese mes";
}
System.out.println("Mes " + mes + ": " + nombreDelMes);
}
}

```

Otra manera de usar case es utilizando llaves, en vez de break, como vemos en el ejemplo siguiente:

En este ejercicio me pide un mes, un día y me devuelve el signo del zodiaco que pertenece

```

import java.util.Scanner;
public class Condicionales10 {
    public static void main (String[] args) {
        Scanner s = new Scanner(System.in);
        String mes,horoscopo;
        int dia;

        System.out.println("Detector de horoscopos! Impresiona a las chicas del horoscopo");
        System.out.print("Introduzca su mes de nacimiento: ");
        mes = s.nextLine();

        switch (mes) {
            case "Enero", "enero", "1" -> {
                System.out.print("Introduzca su día de nacimiento: ");
                dia = s.nextInt();
                if (dia > 0 && dia < 20) {
                    horoscopo = "Capricornio";
                } else if (dia >= 20 && dia <= 31) {
                    horoscopo = "Acuario";
                } else {
                    horoscopo = "Jaja, te crees gracioso?";
                }
            }
            case "Febrero", "febrero", "2" -> {
                System.out.print("Introduzca su día de nacimiento: ");
                dia = s.nextInt();
                if (dia > 0 && dia < 19) {
                    horoscopo = "Acuario";
                } else if (dia >= 19 && dia <= 29) {
                    horoscopo = "Piscis";
                } else {
                    horoscopo = "Jaja, te crees gracioso?";
                }
            }
            case "Marzo", "marzo", "3" -> {
                System.out.print("Introduzca su día de nacimiento: ");
                dia = s.nextInt();
                if (dia > 0 && dia <= 20) {
                    horoscopo = "Piscis";
                } else if (dia > 20 && dia <= 31) {
                    horoscopo = "Aries";
                } else {
                    horoscopo = "Jaja, te crees gracioso?";
                }
            }
        }
    }
}

```

```

case "Abril", "abril", "4" -> {
    System.out.print("Introduzca su día de nacimiento: ");
    dia = s.nextInt();
    if (dia > 0 && dia < 20) {
        horoscopo = "Aries";
    } else if (dia >= 20 && dia <= 30) {
        horoscopo = "Tauro";
    } else {
        horoscopo = "Jaja, te crees gracioso?";
    }
}
case "Mayo", "mayo", "5" -> {
    System.out.print("Introduzca su día de nacimiento: ");
    dia = s.nextInt();
    if (dia > 0 && dia <= 20) {
        horoscopo = "Tauro";
    } else if (dia > 20 && dia <= 31) {
        horoscopo = "Géminis";
    } else {
        horoscopo = "Jaja, te crees gracioso?";
    }
}
case "Junio", "junio", "6" -> {
    System.out.print("Introduzca su día de nacimiento: ");
    dia = s.nextInt();
    if (dia > 0 && dia <= 20) {
        horoscopo = "Géminis";
    } else if (dia > 20 && dia < 31) {
        horoscopo = "Cáncer";
    } else {
        horoscopo = "Jaja, te crees gracioso?";
    }
}
case "Julio", "julio", "7" -> {
    System.out.print("Introduzca su día de nacimiento: ");
    dia = s.nextInt();
    if (dia > 0 && dia < 23) {
        horoscopo = "Cáncer";
    } else if (dia >= 23 && dia <= 31) {
        horoscopo = "Leo";
    } else {
        horoscopo = "Jaja, te crees gracioso?";
    }
}
case "Agosto", "agosto", "8" -> {
    System.out.print("Introduzca su día de nacimiento: ");
    dia = s.nextInt();
    if (dia > 0 && dia < 23) {
        horoscopo = "Leo";
    } else if (dia >= 23 && dia <= 31) {
        horoscopo = "Virgo";
    } else {
        horoscopo = "Jaja, te crees gracioso?";
    }
}
case "Septiembre", "septiembre", "9" -> {
    System.out.print("Introduzca su día de nacimiento: ");

```



```

        dia = s.nextInt();
        if (dia > 0 && dia < 23) {
            horoscopo = "Virgo";
        } else if (dia >= 23 && dia <= 30) {
            horoscopo = "Libra";
        } else {
            horoscopo = "Jaja, te crees gracioso?";
        }
    }
    case "Octubre", "octubre", "10" -> {
        System.out.print("Introduzca su día de nacimiento: ");
        dia = s.nextInt();
        if (dia > 0 && dia < 23) {
            horoscopo = "Libra";
        } else if (dia >= 23 && dia <= 31) {
            horoscopo = "Escorpio";
        } else {
            horoscopo = "Jaja, te crees gracioso?";
        }
    }
    case "Noviembre", "noviembre", "11" -> {
        System.out.print("Introduzca su día de nacimiento: ");
        dia = s.nextInt();
        if (dia > 0 && dia < 22) {
            horoscopo = "Escorpio";
        } else if (dia >= 22 && dia <= 30) {
            horoscopo = "Sagitario";
        } else {
            horoscopo = "Jaja, te crees gracioso?";
        }
    }
    case "Diciembre", "diciembre", "12" -> {
        System.out.print("Introduzca su día de nacimiento: ");
        dia = s.nextInt();
        if (dia > 0 && dia < 22) {
            horoscopo = "Sagitario";
        } else if (dia >= 22 && dia <= 31) {
            horoscopo = "Capricornio";
        } else {
            horoscopo = "Jaja, te crees gracioso?";
        }
    }
    default -> horoscopo = "Por favor introduzca un mes que exista ._.";
}
System.out.println("Tu horoscopo es: " + horoscopo);
s.close();
}
}

```

16 BUCLES.

Bucle for

Se suele utilizar cuando se conoce previamente el número exacto de iteraciones (repeticiones) que se van a realizar. La sintaxis es la siguiente:

```
for (expresion1 ; expresion2 ; expresion3) {  
    sentencias  
}
```

Ejemplo

```
for (int i = 1; i < 11; i++) {  
    System.out.println(i);  
}
```

Ejemplo

tabla de multiplicar con bucles anidados

```
public class Valor {  
    public static void main (String [ ] args) {  
        for (int i=1; i<=10;i++){  
            System.out.println("tabla multiplicar del "+i);  
            for (int a=1;a<=10;a++){  
                System.out.println(i+" x" + a+" = "+i*a);  
            }  
        }  
    }  
}
```

Bucle while

El bucle while se utiliza para repetir un conjunto de sentencias siempre que se cumpla una determinada condición. Es importante reseñar que la condición se comprueba al comienzo del bucle, por lo que se podría dar el caso de que dicho bucle no se ejecutase nunca. La sintaxis es la siguiente:

```
while (expresion) {  
    sentencias  
}
```

Ejemplo

```
int i = 1;  
while (i < 11) {  
    System.out.println(i);  
    i++;  
}
```

Ejemplo

Tabla de multiplicar

```
public class TablaMultiplicar {  
    public static void main (String [ ] args) {  
        int tabla = 1;  
        int multiplicador = 1;  
        int resultado = 0;  
  
        while (tabla <= 10) {  
            resultado = tabla * multiplicador;  
            System.out.println(+tabla + "*" + multiplicador + "=" + ++resultado);  
            multiplicador = multiplicador + 1;  
        }  
  
        if (multiplicador == 11) {  
            System.out.println("Tabla de multiplicar del " + tabla);  
            multiplicador = 1;  
        }  
    }  
}
```

```

        tabla = tabla + 1;
    }
}
}

```

Bucle do-while

El bucle do-while funciona de la misma manera que el bucle while, con la salvedad de que expresión se evalúa al final de la iteración. Las sentencias que encierran el bucle do-while, por tanto, se ejecutan como mínimo una vez. La sintaxis es la siguiente:

```

do {
    sentencias
} while (expresión)

```

Ejemplo

```

int i = 1;
do {
    System.out.println(i);
    i++;
} while (i < 11);

```

17 PAQUETES EN JAVA.

Un Paquete en Java es un contenedor de clases que permite agrupar las distintas partes de un programa y que por lo general tiene una funcionalidad y elementos comunes, definiendo la ubicación de dichas clases en un directorio de estructura jerárquica

Sabías que

Abra VS Code, abra el menú Ver y seleccione Paleta de comando. Escriba SFDX: Crear proyecto con manifiesto en la paleta, y seleccione SFDX: Crear proyecto con manifiesto desde las opciones que aparecen. Seleccione Analytics e ingrese un nombre de proyecto que sea fácil de recordar

En el caso de Eclipse, aquí podemos ver un ejemplo:

```

1 package swing;
2
3 import javax.swing.*;
4
5 public class Ejercicio9TextButton extends JFrame implements ActionListener {
6     private JLabel label1, label2;
7     private JTextField textfield1, textfield2;
8     private JButton boton1;
9     public Ejercicio9TextButton() {
10         setLayout(null);
11         label1 = new JLabel("Nombre:");
12         label1.setBounds(10, 10, 100, 30);
13         add(label1);
14         label2 = new JLabel("Clave:");
15         label2.setBounds(10, 50, 100, 30);
16         add(label2);
17         textfield1 = new JTextField();
18         textfield1.setBounds(130, 10, 100, 30);
19         add(textfield1);
20         textfield2 = new JTextField();
21         textfield2.setBounds(130, 50, 100, 30);
22         add(textfield2);
23         boton1 = new JButton("Confirmar");
24         boton1.setBounds(10, 100, 100, 30);
25         add(boton1);
26         boton1.addActionListener(this);
27     }
28
29     public void actionPerformed(ActionEvent e) {
30         if (e.getSource() == boton1) {

```

18 LIBRERIAS EN JAVA.

Una librería Java se puede entender como un conjunto de clases que facilitan operaciones y tareas ofreciendo al programador funcionalidad ya implementada y lista para ser usada través de una Interfaz de Programación de Aplicaciones, comúnmente abreviada como API (por el anglicismo Application Programming Interface).

- **java.lang:** Contiene la clases fundamentales del lenguaje. Se carga automáticamente con cualquier programa, sin necesidad de hacer import.
- **java.util:** Clases relacionadas con las colecciones y estructuras de datos: listas, colas, pilas, y muchas otras.
- **java.io:** Clases relacionadas con la entrada/salida por consola y por otras vías.
- **java.math:** Clases para cálculos y operaciones matemáticas.
- **java.awt:** Clases para generar interfaces gráficos (ventanas, botones, etc)
- **java.sql:** Clases para gestionar el acceso a bases de datos externas.
- **java.net:** Clases para gestionar el trabajo en red.

18.1 Clase Math.

Con esta clase Podemos usar numerosas funciones matemáticas:

import java.lang.Math. *;

Método.	Descripción.
abs(double a)	Devuelve el valor absoluto de un valor double introducido como parámetro.
abs(float a)	Devuelve el valor absoluto de un valor float introducido como parámetro.
abs(int a)	Devuelve el valor absoluto de un valor Entero introducido como parámetro.
abs(long a)	Devuelve el valor absoluto de un valor long introducido como parámetro.
acos(double a)	Devuelve el arco coseno de un valor introducido como parámetro.
addExact(int x, int y)	Devuelve la suma de sus argumentos, lanzando una excepción si el resultado desborda un int.
addExact(long x, long y)	Devuelve la suma de sus argumentos, lanzando una excepción si el resultado se desborda a long.
asin(double a)	Devuelve el arco seno de un valor introducido.
atan(double a)	Devuelve el arco tangente de un valor introducido.
cbrt(double a)	Devuelve la raíz cúbica de un doublevalor.
cos(double a)	Devuelve el coseno trigonométrico de un ángulo.
exp(double a)	Devuelve el número e de Euler elevado a la potencia de un doublevalor.
log(double a)	Devuelve el logaritmo natural (base e) de un double valor.
log10(double a)	Devuelve el logaritmo de base 10 de un doublevalor.
max(double a, double b)	Devuelve el mayor de dos valores double
max(float a, float b)	Devuelve el mayor de dos valores float.
max(int a, int b)	Devuelve el mayor de dos valores Enteros.
max(long a, long b)	Devuelve el mayor de dos valores long.
min(double a, double b)	Devuelve el menor de dos valores double.
min(float a, float b)	Devuelve el menor de dos valores float.
min(int a, int b)	Devuelve el menor de dos valores enteros.
min(long a, long b)	Devuelve el menor de dos valores long.
multiplyExact(int x, int y)	Devuelve el producto de los argumentos, lanzando una excepción si el resultado desborda un int.
multiplyExact(long x, long y)	Devuelve el producto de los argumentos, lanzando una excepción si el resultado desborda un long.

<code>pow(double a, double b)</code>	Devuelve el valor del primer argumento elevado a la potencia del segundo argumento.
<code>random()</code>	Devuelve un doublevalor con un signo positivo, mayor o igual que 0.0 y menor que 1.0.
<code>round(double a)</code>	Devuelve el long redondeado más cercano al double introducido.
<code>round(float a)</code>	Devuelve el int mas cercano y redondeado al float introducido.
<code>sin(double a)</code>	Devuelve el seno trigonométrico de un ángulo.
<code>sqrt(double a)</code>	Devuelve la raíz cuadrada positiva correctamente redondeada de un doublevalor.
<code>tan(double a)</code>	Devuelve la tangente trigonométrica de un ángulo.

Ejemplos

```
import java.lang.Math;

public class Mates {

    public static void main (String[] args) {

        double numero;
        int potencia;
        numero=12.34;
        potencia=3;
        System.out.println(" su numero es "+Math.round(numero));
        numero=Math.round(numero);
        System.out.println("    su    numero    es    "+Math.round(Math.pow(numero,
potencia)));

    }

}
```

19 GENERACIÓN DE NÚMEROS ALEATORIOS

Esto se realizan con `Math.Random`

Veamos un ejemplo, del juego piedra, papel tijeras

```
import java.lang.Math. *;
import javax.swing.JOptionPane;
public class Aleatorio07 {
    public static void main(String[] args) {
```

```

        int mano = (int)(Math.random()*3); // genera un número al azar
        String eleccion, mensaje;
        mensaje="";
        String name = new String (JOptionPane.showInputDialog("Indique (piedra,
papel, tijeras "));
        switch (mano) {
            case 0 -> {
                eleccion="piedra";

                if (name.equals("papel")){
                    mensaje=" gana a ";

                }
                else if (name.equals("piedra")){
                    mensaje=" empata a ";}

                else if (name.equals("tijeras")){
                    mensaje=" pierde con ";}

                JOptionPane.showMessageDialog(null, "tu has elegido
"+name+mensaje+ eleccion);

            }
            case 1 -> {
                eleccion="papel";

                if (name.equals("papel")){
                    mensaje=" empata a ";

                }
                else if (name.equals("piedra")){
                    mensaje=" pierde con ";}

                else if (name.equals("tijeras")){
                    mensaje=" gana a ";}

                else {
                    mensaje=" error ";

                }

                JOptionPane.showMessageDialog(null, "tu has elegido
"+name+mensaje+ eleccion);

            }
            case 2 -> {
                eleccion="tijeras";
                if (name.equals("papel")){
                    mensaje=" pierde con ";

```

```

    }
    else if (name.equals("piedra")){
        mensaje=" gana a ";}

    else if (name.equals("tijeras")){
        mensaje=" empata con ";}

    else {
        mensaje=" error ";

    }

    JOptionPane.showMessageDialog(null, "tu has elegido
"+name+mensaje+ eleccion);
    }

    }

}
}
}

```

20 2ª EVALUACIÓN

21 ARRAY Y CLASE ARRAY

Un **array** es un tipo de dato capaz de almacenar múltiples valores, vamos a distinguir los arrays unidimensionales o vectores, y los multidimensionales. Las posiciones de los arrays comienzan por 0 y en este ejemplo termina en 3, por tanto son 4 elementos que van desde 0,1,2,3

Ejemplo de definición de un array

```

int[] n; // se define n como un array de enteros
n = new int[4]; // se reserva espacio para 4 enteros
n[0] = 26;
n[1] = -30;
n[2] = 0;
n[3] = 100;

```

otra manera de expresarlo sería:

```

int[] n={26,-30,0,100};

```

En Java los arrays tienen que ser del mismo tipo.

Como se puede recorrer un array: podemos utilizar los bucles, o bien uno especial para array el `for....each()`

Ejemplo de recorrido de un array.
Un array que almacena notas de los alumnos.

```
public class Array05 {  
    public static void main(String[] args) {  
        double[] nota = new double[4];  
        System.out.println("Para calcular la nota media necesito saber la ");  
        System.out.println("nota de cada uno de tus exámenes.");  
        for (int i = 0; i < 4; i++) {  
            System.out.print("Nota del examen nº " + (i + 1) + ": ");  
            nota[i] = Double.parseDouble(System.console().readLine());  
        }  
        System.out.println("Tus notas son: ");  
        double suma = 0;  
        for (int i = 0; i < 4; i++) {  
            System.out.print(nota[i] + " ");  
            suma += nota[i];  
        }  
        System.out.println("\nLa media es " + suma / 4);  
    }  
}
```

Arrays multidimensionales

Se utilizan dos o más índices. En este ejemplo usamos un array bidimensional

Veamos un ejemplo:

```
public class ArrayBi01 {  
    public static void main(String[] args)  
        throws InterruptedException { // Se añade esta línea para poder usar sleep  
        int[][] n = new int[3][2]; // array de 3 filas por 2 columnas  
        n[0][0]=20;  
        n[1][0]=67;  
        n[1][1]=33;  
        n[2][1]=7;  
        int fila, columna;  
        for(fila = 0; fila < 3; fila++) {  
            System.out.print("Fila: " + fila);  
            for(columna = 0; columna < 2; columna++) {  
                System.out.printf("%10d ", n[fila][columna]);  
                Thread.sleep(1000); // retardo de un segundo  
            }  
            System.out.println();  
        }  
    }  
}
```

Array tridimensional

Si quisiéramos crear un array tridimensional una manera de hacerlo sería:

```
String[][][] datos = {
    {
        {"Alberto", "10", "Programación"},
        {"Ana", "8", "Base de datos"},
        {"Eva", "9", "Entorno de desarrollo"}
    }

    {
        {"enero", "202", "pagado"},
        {"febrero", "280", "pagado"},
        {"marzo", "209", "pagado"}
    }
};
```

Saber más

<https://codesitio.com/recursos-utiles-para-tu-web-o-blog/cursos/curso-de-java-matrices-multidimensionales/>

Recorrer arrays con for al estilo foreach

Hace un recorrido sobre todo el array, veamos un ejemplo:

```
package array;
import java.util.Scanner;
public class ArrayForEach {
    public static void main(String[] args) {
        Scanner s=new Scanner(System.in);
        double[] nota = new double[4];
        System.out.println("Para calcular la nota media
necesito saber la ");
        System.out.println("nota de cada uno de tus
exámenes.");
        for (int i = 0; i < 4; i++) {
            System.out.print("Nota del examen nº " + (i +
1) + ": ");
            nota[i] = Double.parseDouble(s.nextLine());
        }
        System.out.println("Tus notas son: ");
        double suma = 0;
        for (double y : nota) { // for al estilo foreach
            System.out.print(y + " ");
            suma += y;
        }
        System.out.println("\nLa media es " + suma / 4);
    }
}
```

```
}  
}
```

En este ejemplo la variable `n`, es igual al array, de manera que el array de una manera sencilla, me muestra las notas sacadas.

Ejemplo con `foreach` anidados para arrays multidimensionales

Para recorrer un array multidimensional en Java utilizando un ciclo `foreach`, puedes utilizar un ciclo anidado. El ciclo externo se utiliza para recorrer cada uno de los elementos del primer nivel del array, mientras que el ciclo interno se utiliza para recorrer cada uno de los elementos del segundo nivel del array.

Aquí tienes un ejemplo de cómo recorrer el array que mencionas utilizando un ciclo `foreach`:

```
int[][] array = {{2,3}, {5,6}, {9,10}};
```

```
for (int[] subArray : array) {  
    for (int element : subArray) {  
        System.out.print(element + " ");  
    }  
    System.out.println();  
}
```

En este ejemplo, el ciclo externo recorre cada uno de los elementos del primer nivel del array (cada sub-array) y el ciclo interno recorre cada uno de los elementos del segundo nivel del array (cada elemento dentro de cada sub-array).

El código imprimirá en pantalla:

Copy code

```
2 3  
5 6  
9 10
```

21.1 La clase `Array`.

Existe una clase denominada `Arrays` que nos permite acceder a los arrays y manipularlos de una manera más eficiente. Para ello debemos importar la clase `Arrays`:

```
import java.util.Arrays;
```

- [Búsqueda de elementos](#)

Podemos utilizar para ello el método **`indexOf`** que nos va a permitir que nos indique cual es la posición del elemento buscado en el array como vemos en el ejemplo:

```
import java.util.Arrays;  
public class Array1 {  
  
    public static void main (String[] args) {  
        int index=0;
```

```

String[] datos = {"Alberto", "Ana", "Eva"};
index = Arrays.asList(datos).indexOf("Ana");

if (index >= 0) {
    System.out.println("encontrado el indices es " + index);
} else {
    System.out.println("no encontrado " + index);
}

}
}

```

- **Mostrar los datos de un array**

Para ello nos valemos del método toString, como vemos en el ejemplo siguiente:

```

String[] names = {"Alberto", "Ana", "Eva"};
System.out.println(Arrays.toString(names));

```

- **Ordenar datos de un array**

Para ello utilizamos el método sort, que ordena ascendentemente y el método reverse lo hace descendientemente

Ejemplo con sort

```

import java.util.Arrays;

public class Array2 {
    public static void main(String[] args) {
        String[] names = {"Eva", "Laura", "Alberto"};
        Arrays.sort(names);
        System.out.println(Arrays.toString(names)); // Imprime "[Alberto, Eva, Laura]"
    }
}

```

Ejemplo con reverse

```

import java.util.Arrays;
import java.util.Collections;

public class Array3 {
    public static void main(String[] args) {
        String[] names = {"Eva", "Laura", "Alberto"};
        Arrays.sort(names);
        Collections.reverse(Arrays.asList(names));
        System.out.println(Arrays.toString(names)); // Imprime "[Eva, Laura, Alberto]"
    }
}

```

Ejemplo en donde copiamos el resultado de la ordenación de un array en otro array

```
import java.util.Arrays;

public class Array4 {
    public static void main(String[] args) {
        String[] names = {"Eva", "Laura", "Alberto"};
        Arrays.sort(names);
        String[] sortedNames = Arrays.copyOf(names, names.length);
        System.out.println(Arrays.toString(sortedNames)); // Imprime "[Alberto, Eva, Laura]"
    }
}
```

El método `sort()` ordena el array original en orden alfabético, y luego el método `copyOf()` copia los elementos del array original en otro array llamado `sortedNames`.

Si quieres ordenar el array en orden inverso, puedes utilizar el método `reverse()` de la clase `Arrays`, como se muestra a continuación:

```
import java.util.Arrays;

public class Array5 {
    public static void main(String[] args) {
        String[] names = {"Eva", "Laura", "Alberto"};
        Arrays.sort(names);
        Arrays.reverse(names);
        String[] sortedNames = Arrays.copyOf(names, names.length);
        System.out.println(Arrays.toString(sortedNames)); // Imprime "[Laura, Eva, Alberto]"
    }
}
```

- Comparar el contenido de dos arrays

Para comparar si dos arrays tienen los mismos datos en Java, puedes utilizar el método `equals()` de la clase `Arrays`.

Aquí tienes un ejemplo de cómo hacerlo:

```
import java.util.Arrays;

public class Array6 {
    public static void main(String[] args) {
        int[] array1 = {1, 2, 3};
        int[] array2 = {1, 2, 3};
    }
}
```

```
boolean areEqual = Arrays.equals(array1, array2);  
System.out.println(areEqual); // Imprime "true"  
}  
}
```

El método equals() compara los elementos de ambos arrays y devuelve true si todos los elementos son iguales, y false en caso contrario.

Si quieres comparar arrays de tipos de datos distintos a int, puedes utilizar el método equals() de la clase Arrays correspondiente, por ejemplo Arrays.equals(Object[], Object[]) para arrays de objetos o Arrays.equals(boolean[], boolean[]) para arrays de booleanos.

Para comparar si dos arrays tienen los mismos datos en Java, puedes utilizar el método equals() de la clase Arrays.

Aquí tienes un ejemplo de cómo hacerlo:

```
import java.util.Arrays;  
  
public class Array7 {  
    public static void main(String[] args) {  
        String[] array1 = {"Alberto", "Eva", "Laura"};  
        String[] array2 = {"Laura", "Eva", "Alberto"};  
        boolean areEqual = Arrays.equals(array1, array2);  
        System.out.println(areEqual); // Imprime "false"  
    }  
}
```

El método equals() compara los elementos de ambos arrays y devuelve true si todos los elementos son iguales, y false en caso contrario.

En este caso, el resultado sería false ya que aunque los elementos del array son los mismos, están en diferente orden.

Si quieres comparar los arrays independientemente del orden de los elementos, puedes ordenar ambos arrays y luego compararlos con el método equals(). Por ejemplo:

```
import java.util.Arrays;
```

```

public class Array8 {
    public static void main(String[] args) {
        String[] array1 = {"Alberto", "Eva", "Laura"};
        String[] array2 = {"Laura", "Eva", "Alberto"};
        Arrays.sort(array1);
        Arrays.sort(array2);
        boolean areEqual = Arrays.equals(array1, array2);
        System.out.println(areEqual); // Imprime "true"
    }
}

```

De esta manera, el resultado sería true ya que ambos arrays están ordenados y tienen los mismos elementos.

- Rellenar un array con un valor

Para ello usamos el método fill, lo que hacemos es rellenar un array con un mismo valor.

```

import java.util.Arrays;

public class Array9{
    public static void main(String[] args) {
        String[] array1 = new String[5];
        for (int i = 0; i < array1.length; i++) {
            Arrays.fill(array1, "Alberto");
        }

        System.out.println(Arrays.toString(array1)); // Imprime "[Alberto, Alberto, Alberto, Alberto, Alberto]"
    }
}

```

- Clonar un array

Devuelve una copia exacta del array, en el caso de datos primitivos la copia no afecta al original, es decir son arrays totalmente diferentes. De manera que si creamos

array1 [1]="Cristina"; el resultado cambiará en el array1, pero no se verá afectado el array2

```
import java.util.Arrays;

public class Array10{
    public static void main(String[] args) {
        String[] array1 = {"Alberto", "Eva", "Laura"};
        String[] array2 = array1.clone();
        System.out.println(Arrays.toString(array2)); // Imprime "[Alberto, Eva, Laura]"
    }
}
```

En este ejemplo al no ser cloneable, un array multidimensional, ocurre que al cambiar un valor de un array se cambia en el otro:

```
import java.util.Arrays;

public class Array1 {
    public static void main(String[] args) {
        int[][] array1 = {{2, 5, 23, 15, 4}, {36, 1, 87, 0, 12}};
        int[][] array2 = array1.clone();
        array1[1][1] = 44;
        System.out.println(Arrays.deepToString(array1)); // Imprime "[[2, 5, 23, 15, 4], [36, 44, 87, 0, 12]]"
        System.out.println(Arrays.deepToString(array2)); // Imprime "[[2, 5, 23, 15, 4], [36, 44, 87, 0, 12]]"
    }
}
```

- Métodos `Arrays.copyOf` y `Arrays.copyOfRange`.

Ya hemos visto al principio como realizar una copia de un array a otro, ahora vamos a ver el método `Arrays.copyOfRange`.

```
import java.util.Arrays;

public class Array11 {
```



```

public static void main(String[] args) {
    int[] array1 = {1, 2, 3, 4, 5};
    int[] array2 = Arrays.copyOfRange(array1, 1, 3);
    System.out.println(Arrays.toString(array2)); // Imprime "[2, 3]"
}
}

```

El método `copyOfRange()` toma tres argumentos: el array original, el índice inicial y el índice final del rango a copiar. En este caso, se copian los elementos del índice 1 al índice 3 (sin incluir el índice 3) del array `array1` en el array `array2`.

- Insertar elementos en un array

Para insertar un elemento en un array en Java, puedes utilizar un bucle para desplazar los elementos del array a la derecha a partir del índice en el que quieres insertar el nuevo elemento.

Aquí tienes un ejemplo de cómo insertar el elemento "Juan" en el array `array1` después del elemento "Alberto":

```

import java.util.Arrays;

public class Array12{

    public static void main(String[] args) {
        String[] array1 = {"Alberto", "Eva", "Laura"};
        String[] array2 = new String[array1.length + 1];
        for (int i = 0; i < array1.length; i++) {
            if (i == 0) {
                array2[i] = "Juan";
            }
            array2[i + 1] = array1[i];
        }
        System.out.println(Arrays.toString(array2)); // Imprime "[Juan, Alberto, Eva, Laura]"
    }
}

```

En este caso, se crea un nuevo array `array2` con una posición más que el array original. Luego, se utiliza un bucle para recorrer el array `array1` y asignar cada elemento al nuevo array, excepto el primer elemento, que se reemplaza por el elemento "Juan".

El resultado sería un array `array2` con los elementos "[Juan, Alberto, Eva, Laura]"

- Eliminar elementos de un array

Para eliminar un elemento de un array en Java, puedes utilizar un bucle para recorrer el array y copiar los elementos en otro array, saltándote el elemento que quieres eliminar.

Aquí tienes un ejemplo de cómo eliminar el elemento "Juan" del array array1:

```
import java.util.Arrays;

public class Array13 {

    public static void main(String[] args) {

        String[] array1 = {"Juan", "Alberto", "Eva", "Laura"};

        String[] array2 = new String[array1.length - 1];

        for (int i = 0, j = 0; i < array1.length; i++) {

            if (!array1[i].equals("Juan")) {

                array2[j] = array1[i];

                j++;

            }

        }

        System.out.println(Arrays.toString(array2)); // Imprime "[Alberto, Eva, Laura]"

    }

}
```

En este caso, se crea un nuevo array array2 con una posición menos que el array original. Luego, se utiliza un bucle para recorrer el array array1 y copiar los elementos en el nuevo array, excepto el elemento "Juan".

El resultado sería un array array2 con los elementos "[Alberto, Eva, Laura]".

Si queremos anular un array, no existe forma de eliminarlo de la memoria, pero si de darle un valor null

Por ejemplo

```
array1=null;
```

22 MÉTODOS O FUNCIONES

Un método es una función que, al mismo tiempo, forma parte de una clase que puede realizar operaciones sobre datos de esa misma clase. En Java, un programa consta solo de clases y las funciones solo se pueden describir dentro de ellas. Por eso todas las funciones del lenguaje Java son métodos.

Veamos un ejemplo

Aquí tienes un ejemplo de cómo realizar una multiplicación mediante una función en Java:

```
public class Ejercicio1 {
    public static int multiply(int a, int b) {
        return a * b;
    }

    public static void main(String[] args) {
        int result = multiply(5, 7);
        System.out.println(result); // Imprime "35"
    }
}
```

En este caso, se define la función `multiply()` que toma dos argumentos de tipo `int` y devuelve el resultado de multiplicar ambos números. Luego, se llama a la función desde el método `main()` y se imprime el resultado. Escribimos `static` para no tener que necesitar instanciar un objeto.

La palabra reservada `static` permite diferenciar entre un método que no tiene porque ser instanciado y sin `static`, tendríamos que crear una clase de ese objeto, veamos un ejemplo.

- Ejemplo de calculadora con método `static`

```
//ejemplo de calculadora sin creacion de objetos por ello usamos metodos
static
import java.util.Scanner;
public class Calculadora {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);

        System.out.print("indica el primer número; ");
        int a=s.nextInt();
        System.out.print("indica el segundor número; ");
        int b=s.nextInt();
        //invocamos los diferentes metodos
        System.out.println("Suma: " + suma(a, b));
        System.out.println("Resta: " + resta(a, b));
        System.out.println("Producto: " + producto(a, b));
        System.out.println("División: " + division(a, b));
    }
    // métodos static, no necesitan instanciar una clase.
    public static int suma(int a, int b) {
        return a + b;
    }

    public static int resta(int a, int b) {
```

```

        return a - b;
    }

    public static int producto(int a, int b) {
        return a * b;
    }

    public static int division(int a, int b) {
        return a / b;
    }
}

```

- Calculadora con objetos y métodos.

package metodos;

// al no usar static, tenemos que crear un objeto de la clase y usamos los metodos sin static

```

public class CalculadoraObjeto {
    public static void main(String[] args) {
        CalculadoraObjeto calculator = new CalculadoraObjeto();
        int a = 10, b = 5;
        System.out.println("Suma: " + calculator.Suma(a, b));
        System.out.println("Resta: " + calculator.Resta(a, b));
        System.out.println("Producto: " + calculator.Producto(a, b));
        System.out.println("División: " + calculator.Division(a, b));
    }

    public int Suma(int a, int b) {
        return a + b;
    }

    public int Resta(int a, int b) {
        return a - b;
    }

    public int Producto(int a, int b) {
        return a * b;
    }

    public int Division(int a, int b) {
        return a / b;
    }
}

```

Creamos un paquete denominado calculadora, en ella vamos a crear una clase con este código.

package calculadora;

```

import calculadora.funciones.FuncionesMath.*;

import java.util.Scanner;
public class Calculo {

    //ejemplo de calculadora sin creacion de objetos por ello usamos
    metodos static

    public static void main(String[] args) {
        Scanner s= new Scanner(System.in);

        //hemos creado un objeto para realizar otro ejemplo, basado en
        la clase FuncionesMath

        System.out.print("indica el primer número; ");
        int a=s.nextInt();
        System.out.print("indica el segundor número; ");
        int b=s.nextInt();

        //invocamos los diferentes metodos
        System.out.println("Suma: " +
        calculadora.funciones.FuncionesMath.suma(a, b));
        System.out.println("Suma: " +
        calculadora.funciones.FuncionesMath.resta(a, b));
        System.out.println("Suma: " +
        calculadora.funciones.FuncionesMath.producto(a, b));
        System.out.println("Suma: " +
        calculadora.funciones.FuncionesMath.division(a, b));

    }

}

```

Creamos una carpeta dentro del paquete calculadora que se llama FuncionesMath con este código.

```

package calculadora.funciones;

public class FuncionesMath {
    // métodos static, no necesitan instanciar una clase.
    public static int suma(int a, int b) {
        return a + b;
    }
    public static int resta(int a, int b) {
        return a - b;
    }
}

```

```

        public static int producto(int a, int b) {
            return a * b;
        }

        public static int division(int a, int b) {
            return a / b;
        }
    }
}

```

En este ejemplo hemos utilizado static, por tanto no tendremos que instanciar la clase FuncionesMath,

Realiza el mismo ejemplo pero teniendo que crear un objeto, por tanto los métodos de la clase FuncionesMath habría que quitarles Static, lo llamaremos calculo2 y la clase FuncionesMath2

Podemos crear una sobrecarga de métodos, es decir un nuevo método con el mismo nombre pero diferentes argumentos.

```

public static int suma(int a) {
    return a + a;
}

```

Podemos invocar a un método u otro, todo dependerá del número de argumentos que pongamos en él.

Paso por argumentos o por referencia

En Java, cuando se pasa un argumento a un método, se está pasando una copia del valor del argumento. Esto significa que cualquier cambio realizado al argumento dentro del método no afectará al valor original del argumento fuera del método.

Por otro lado, cuando se pasa una referencia a un objeto a un método, se está pasando una copia de la dirección de memoria del objeto. Esto significa que cualquier cambio realizado al objeto dentro del método se reflejará en el objeto original fuera del método.

En Java, el método "suma" que se describe se comportaría de la misma manera si se pasan los argumentos "a" y "b" por valor o por referencia. Ya que los argumentos son de tipo primitivo (int), se pasan por valor automáticamente, y cualquier cambio realizado en los argumentos dentro del método no afectará los valores originales fuera del método.

Aquí te muestro un ejemplo de como quedaría el metodo suma si se pasara por argumento:

```
int suma(int a, int b) {  
    return a + b;  
}
```

Y aquí te muestro como quedaria si se pasara por referencia.

Sin embargo, si quisieras pasar los argumentos por **referencia**, podrías hacerlo utilizando una clase que contenga los atributos "a" y "b", y pasar una instancia de esa clase al método en lugar de pasar los argumentos individualmente.

Aquí te muestro un ejemplo:

```
class Numero {  
    int a;  
    int b;  
}  
  
int suma(Numero num) {  
    return num.a + num.b;  
}
```

En este caso se esta pasando un objeto de la clase Numero, y se accede a los atributos a y b dentro del metodo para realizar la operación de suma.

Nota

Existe una herramienta denominada javadoc, para realizar comentarios.

Etiquetas como

@author, indica el autor

@param para indicar los parámetros de una función

@return especificamos que devuelve

- Creación de bibliotecas de rutinas mediante paquetes.

Las funciones se pueden reutilizar y para ello es mejor crear un paquete que luego se importará desde el programa que necesitemos estas funciones. Cada paquete debe estar en un directorio por ejemplo el paquete matemáticas debe estar en un directorio denominado matemáticas. A su

vez podremos crear subpaquetes o subdirectorios, por ejemplo, para dividir entre geometría y cálculo.

Otra manera de agrupar las funciones en un mismo paquete es creando dos ficheros java diferentes.

Veamos un ejemplo:

Para crear un paquete denominado calculadora con una clase Calculadora que contenga métodos para realizar operaciones matemáticas básicas, puedes hacer lo siguiente:

Crea un nuevo archivo de texto y agrega lo siguiente:

```
package calculadora.funciones;

public class FuncionesMath {

    // métodos static, no necesitan instanciar una clase.
    public static int suma(int a, int b) {
        return a + b;
    }

    public static int resta(int a, int b) {
        return a - b;
    }

    public static int producto(int a, int b) {
        return a * b;
    }

    public static int division(int a, int b) {
        return a / b;
    }

    //sobrecarga de método, podemos crear un método en este caso, para
    instanciar esta clase
```



```

        // con otro criterio
        public int suma(int a) {
            return a+a;

        }
    }
}

```

Guarda el archivo como Calculadora.java en un directorio llamado calculadora.

Compila el archivo con el comando `javac Calculadora.java`. Esto creará un archivo `Calculadora.class` en el directorio calculadora.

Ahora puedes importar el paquete calculadora y usar la clase Calculadora en cualquier otro programa de Java. Por ejemplo:

```

import calculadora.funciones.FuncionesMath;

import java.util.Scanner;

public class Calculo {
    public static void main(String[] args) {
        Scanner s=new Scanner(System.in);
        System.out.print("indica el primer número; ");
        int a=s.nextInt();
        System.out.print("indica el segundor número; ");
        int b=s.nextInt();
        int suma = FuncionesMath.suma(a, b);
        int producto = FuncionesMath.producto(a, b);
        int diferencia = FuncionesMath.resta(a, b);
        int cociente = FuncionesMath.division(a, b);

        System.out.println("Suma: " + suma);
        System.out.println("Producto: " + producto);
    }
}

```

```

System.out.println("Diferencia: " + diferencia);
System.out.println("Cociente: " + cociente);
s.close();
}
}

```

Cuando ejecutes este programa, se imprimirán los resultados de las operaciones matemáticas.

- Paso de parámetros por valor y por referencia

La diferencia estriba que el paso por valor no se modifica el valor en la función pero no el programa, en cambio cuando se pasa por referencia aunque en java no existe como tal el paso por referencia, se puede simular, si veamos un ejemplo primero de paso por parametros

```

public class PruebaParametros1 {
public static void main(String[] args) {
int n = 10;
System.out.println(n);
calcula(n);
System.out.println(n);
}
public static void calcula(int x) {
x += 24;
System.out.println(x);
}
}

```

La salida del programa anterior es la siguiente:

```

10
34
10

```

Aquí tenemos otro ejemplo usando paso por referencia:

```

public class PruebaParametrosArray {
public static void main(String[] args) {
int n[] = {8, 33, 200, 150, 11};
int m[] = new int[5];
muestraArray(n);
incrementa(n);
}
}

```

```

muestraArray(n);
}

public static void muestraArray(int x[]) {
for (int i = 0; i < x.length; i++) {
System.out.print(x[i] + " ");
}
System.out.println();
}

public static void incrementa(int x[]) {
for (int i = 0; i < x.length; i++) {
x[i]++;
}
}
}

```

En este ejemplo estamos usando un constructor y estamos creando un objeto p de clase punto, en donde además estamos utilizando this this es una palabra clave en Java que se utiliza para hacer referencia al objeto actual de una clase.

Creemos una clase denominada Punto, al cual llamamos Punto.java

```

public class Punto {
    public int x;
    public int y;

    public Punto(int x, int y) {
        this.x = x;
        this.y = y;
    }
}

```

Creemos otra clase denominada PruebaReferencia.java

```

public class PruebaReferencia {
    public static void main(String[] args) {
        Punto p = new Punto(1, 2);
        System.out.println("Antes de modificar: " + p.x + ", " + p.y);
    }
}

```

```

    modificarPunto(p);
    System.out.println("Después de modificar: " + p.x + ", " + p.y);
}

public static void modificarPunto(Punto p) {
    p.x = 10;
    p.y = 20;
}
}

```

22.1 Uso de métodos void.

Void significa que no devuelve ningún valor, es lo que en programación tradicional denominamos procedimientos.

Si definimos un método del tipo public void, significa que necesitamos instanciar la clase, en cambio si usamos public static void, significa que no debemos instanciar la clase:

Veamos un ejemplo:

```

import java.util.Scanner;
public class MetodoVoid {
    public static void pedirdatos() {
        Scanner s= new Scanner(System.in);
        System.out.println("Ingrese el primer valor:");
        int a=s.nextInt();
        System.out.println("Ingrese el segundo valor valor:");
        int b=s.nextInt();
        System.out.println(a+b);
    }
    public static void main(String[] args) {
        pedirdatos();
    }
}

```

Hemos creado un método void, que realiza una serie de acciones y no me devuelve ningún valor.

23 PROGRAMACIÓN ORIENTADO A OBJETOS (POO)

Se base en la utilización de objetos, estos se denominan instancias.

Clase

Concepto abstracto que denota una serie de cualidades, por ejemplo **coche**.

Instancia

Objeto palpable, que se deriva de la concreción de una clase, por ejemplo **mi coche**.

Atributos

Conjunto de características que comparten los objetos de una clase, por ejemplo para la clase **coche** tendríamos **matrícula**, **marca**, **modelo**, **color** y **número de plazas**.

Veamos un ejemplo: creamos una clase denominada libro

```
/**
 * Libro.java
 * Definición de la clase Libro
 * @author Alberto Ruiz
 */
public class Libro {
    // atributos
    String isbn;
    String titulo;
    String autor;
    int numeroDePaginas;
}
```

Creamos otra clase en donde crearemos una serie de objetos de la clase libro y le daremos valores, esta clase se denominará PruebaLibro

```
/**
 * PruebaLibro.java
 * Programa que prueba la clase Libro
 * @author Alberto Ruiz
 */
public class PruebaLibro {
    public static void main(String[] args) {
        Libro libro1 = new Libro();
        Libro libro2 = new Libro();
        Libro libro3 = new Libro();
    }
}
```

```

//dar valores al objeto libro1
libro1.autor="Tolkien";
libro1.isbn="1";
libro1.numeroDePaginas=200;
libro1.titulo="El Hobbit";

//dar valores al objeto libro2
libro2.autor="Frank Herbert";
libro2.isbn="2";
libro2.numeroDePaginas=400;
libro2.titulo="Dune";

//dar valores al objeto libro3
libro3.autor="Cervantes";
libro3.isbn="3";
libro3.numeroDePaginas=1200;
libro3.titulo="El Quijote";

// mostrar valores de los objetos
System.out.println(libro1.titulo+" "+libro1.autor+" "+
libro1.isbn+" "+ libro1.numeroDePaginas);
System.out.println(libro2.titulo+" "+libro2.autor+" "+
libro2.isbn+" "+ libro2.numeroDePaginas);
System.out.println(libro3.titulo+" "+libro3.autor+" "+
libro3.isbn+" "+ libro3.numeroDePaginas);
    }
}

```

23.1 Encapsulamiento y ocultación

Encapsulamiento: define todas las propiedades y comportamiento de una clase dentro de esa clase.

Ocultación: permite esconder los elementos que definen una clase, el interior, las tripas.

23.2 Métodos

Son acciones asociadas a una clase

En este ejemplo vamos a añadir un concepto del que hablaremos más adelante que es la herencia. Este es un concepto que define

Solamente el método operar es distinto para las clases Suma y Resta (esto hace que no lo podamos disponer en la clase Operacion), luego los métodos cargar1, cargar2 y

mostrarResultado son idénticos a las dos clases, esto hace que podamos disponerlos en la clase Operacion. Lo mismo los atributos valor1, valor2 y resultado se definirán en la clase padre Operacion.

Crear un proyecto y luego crear cuatro clases llamadas: Operacion, Suma, Resta y Prueba

La herencia significa que se pueden crear nuevas clases partiendo de clases existentes, que tendrá todas los atributos y los métodos de su 'superclase' o 'clase padre' y además se le podrán añadir otros atributos y métodos propios.

Clase operacion

```
import java.util.Scanner;
public class Operacion {
    protected Scanner teclado;
    protected int valor1;
    protected int valor2;
    protected int resultado;
    public Operacion() {
        teclado=new Scanner(System.in);
    }

    public void cargar1() {
        System.out.print("Ingrese el primer valor:");
        valor1=teclado.nextInt();
    }

    public void cargar2() {
        System.out.print("Ingrese el segundo valor:");
        valor2=teclado.nextInt();
    }

    public void mostrarResultado() {
        System.out.println(resultado);
    }
}
```

Clase Suma

```
public class Suma extends Operacion{
    void operar() {
        resultado=valor1+valor2;
    }
}
```

Clase Resta

```
public class Resta extends Operacion {  
    void operar(){  
        resultado=valor1-valor2;  
    }  
}
```

Clase Prueba

```
public class Prueba {  
    public static void main(String[] ar) {  
        Suma suma1=new Suma();  
        suma1.cargar1();  
        suma1.cargar2();  
        suma1.operar();  
        System.out.print("El resultado de la suma es:");  
        suma1.mostrarResultado();  
        Resta resta1=new Resta();  
        resta1.cargar1();  
        resta1.cargar2();  
        resta1.operar();  
        System.out.print("El resultado de la resta  
es:");  
        resta1.mostrarResultado();  
    }  
}
```

23.3 Método constructor

En Java podemos definir un método que se ejecute inicialmente y en forma automática. Este método se lo llama constructor.

El constructor tiene las siguientes características:

- Tiene el mismo nombre de la clase.
- Es el primer método que se ejecuta.
- Se ejecuta en forma automática.
- No puede retornar datos.
- Se ejecuta una única vez.
- Un constructor tiene por objetivo inicializar atributos.

```
import java.util.Scanner;
```



```

public class Operarios {
    private Scanner teclado;
    private int[] sueldos;

    public Operarios()
    {
        teclado=new Scanner(System.in);
        sueldos=new int[5];
        for(int f=0;f<5;f++) {
            System.out.print("Ingrese valor sueldo:");
            sueldos[f]=teclado.nextInt();
        }
    }

    public void imprimir() {
        for(int f=0;f<5;f++) {
            System.out.println(sueldos[f]);
        }
    }

    public static void main(String[] ar) {
        Operarios op=new Operarios();
        op.imprimir();
    }
}

```

Ejercicio

Realiza el mismo ejercicio, pero utilizando para ello, una clase denominado película en donde definiremos los siguientes atributos, nombre, director, duración, año.

Crearemos tres objetos de esta clase película en otra clase denominado ListaPeliculas y le daremos los valores a elegir por el alumno.

23.4 Método Getter/Setter

En Java, los métodos getter y setter son utilizados para obtener (get) y establecer (set) el valor de una propiedad o atributo de una clase. Los métodos getter devuelven el valor de la propiedad, mientras que los métodos setter establecen el valor de la propiedad. Estos métodos se utilizan para encapsular (ocultar) los detalles de implementación de una clase y proporcionar una interfaz pública para acceder y modificar los valores de sus atributos.

```

class Persona {
    private String nombre;
    private int edad;

    // Método getter para el atributo 'nombre'
    public String getNombre() {
        return nombre;
    }

    // Método setter para el atributo 'nombre'
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    // Método getter para el atributo 'edad'
    public int getEdad() {
        return edad;
    }

    // Método setter para el atributo 'edad'
    public void setEdad(int edad) {
        this.edad = edad;
    }
}

```

En este ejemplo, la clase Persona tiene dos atributos privados: nombre y edad. Los métodos getNombre() y getEdad() son los métodos getter correspondientes, mientras que los métodos setNombre(String nombre) y setEdad(int edad) son los métodos setter correspondientes. Los métodos setter establecen el valor del atributo correspondiente, mientras que los métodos getter devuelven el valor del atributo correspondiente.

Puede acceder y modificar los valores de los atributos de la siguiente manera:

```

package objetopersona;
import java.util.Objects;

```

```

import objetopersona.Persona;
public class MainPersona {

    public static void main(String[] args) {
        // TODO Esbozo de método generado automáticamente
        Persona person= new Persona();
        person.setEdad(25);
        person.setNombre("Alberto");

        String MostrarNombre=person.getNombre();
        int MostrarEdad=person.getEdad();

        System.out.println("Su nombres es "+MostrarNombre + " su
edad es "+MostrarEdad);

    }
}

```

23.5 Enum

Definimos tipos enumerados

```

public enum DíasDeLaSemana {

    LUNES,
    MARTES,
    MIÉRCOLES,
    JUEVES,
    VIERNES,
    SÁBADO,
    DOMINGO
}

public class Main {

    public static void main(String[] args) {

        DíasDeLaSemana hoy = DíasDeLaSemana.VIERNES;

        System.out.println("Hoy es: " + hoy);

    }

}

```

Este programa define una enumeración `DíasDeLaSemana` con 7 valores, uno para cada día de la semana. En el método `main`, se crea una variable `hoy` de tipo `DíasDeLaSemana` y se le asigna el valor `VIERNES`. El programa luego imprime el valor de `hoy`. La salida será:

Hoy es: VIERNES

23.6 Clases abstractas

No tendrá instancias de manera directa, pero si las subclases. Para ello utilizaremos la palabra `extends`.

Ejemplo:

Creamos tres clases, una la clase abstracta `Sueldos`, otra la clase hija `empleados`, en la que calculamos un incremento del salario del 10% por ultimo una clase `SalariosMain` en donde ejecutamos la aplicación.

Usaremos la palabra clave `super`, que hace referencia a la clase padre.

`@override` en java, es una anotación para indica que un método en una clase hija esta redefiniendo o anulando un método de la clase padre.

Clase `Sueldos`

```
public abstract class Sueldos {  
    private String nombre;  
    private String puesto;  
    private double salario;  
  
    public Sueldos(String nombre, String puesto, double salario) {  
        this.nombre = nombre;  
        this.puesto = puesto;  
        this.salario = salario;  
    }  
  
    public abstract double calcularSalario();  
    return 0;  
}  
  
    public String getNombre() {  
        return nombre;  
    }  
}
```

```

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public String getPuesto() {
    return puesto;
}

public void setPuesto(String puesto) {
    this.puesto = puesto;
}

public double getSalario() {
    return salario;
}

public void setSalario(double salario) {
    this.salario = salario;
}
}

```

Clase Empleados

```

public class Empleado extends Sueldos {
    public Empleado(String nombre, String puesto, double salario) {
        super(nombre, puesto, salario);
    }
}

```

```

@Override
public double calcularSalario() {
    return getSalario() * 0.10;
}
}

```

SueldosMain

```
public class SueldosMain {  
    public static void main(String[] args) {  
        Sueldos empleado = new Empleado("Juan", "Programador", 5000.0);  
        System.out.println("El Salario del " + empleado.getNombre() + " es: " +  
empleado.calcularSalario());  
    }  
}
```

En este ejemplo, la clase Sueldos está definida como abstracta y tiene tres variables de instancia nombre, puesto y salario con métodos getter y setter para cada uno. La clase también incluye un método abstracto calcularBono() que debe ser implementado por cualquier clase concreta que extienda Sueldos. La clase Empleado es una clase hija que extiende a Sueldos y implementa el método abstracto calcularBono(). La clase Main crea un objeto empleado de tipo Sueldos y lo asigna a una instancia de la clase Empleado. Luego, el programa imprime el bono del empleado.

Ejercicio1

Realiza el mismo ejercicio en el que se escriba nombre y apellidos de un alumno, la asignatura y la nota, y según la nota diga si es suspenso o aprobado.

Ejercicio2

Realiza el mismo ejercicio en el que se escriba un nombre de usuario y una contraseña que este almacenada en un array, y que si se escribe el nombre de usuario y contraseña y no sean los que están en el array de objetos, de un mensaje de error, debe dar tres oportunidades.

Ejemplo

En el caso de que mezclemos ambos ejercicios este sería el resultado

```
import java.util.Scanner;  
  
abstract class Notas {  
    String nombreAsignatura;  
    int nota;  
  
    public Notas(String nombreAsignatura, int nota) {  
        this.nombreAsignatura = nombreAsignatura;  
        this.nota = nota;  
    }  
}
```

```

    abstract int calcularNotaFinal();
}

class Alumno extends Notas {
    String nombre;
    int[] notas;

    public Alumno(String nombre, int[] notas) {
        super("", 0);
        this.nombre = nombre;
        this.notas = notas;
    }

    int calcularNotaFinal() {
        int sumaNotas = 0;
        for (int nota : notas) {
            sumaNotas += nota;
        }
        return sumaNotas / notas.length;
    }
}

class AlumnoMain {
    static String[][] usuarios = {{"alberto", "1234"}, {"ana", "5678"}};

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int intentos = 3;
        while (intentos > 0) {
            System.out.print("Introduce tu nombre de usuario: ");
            String usuario = sc.nextLine();
            System.out.print("Introduce tu contraseña: ");

```

```

String contraseña = sc.nextLine();
boolean autenticado = false;
for (String[] datosUsuario : usuarios) {
    if (usuario.equals(datosUsuario[0]) && contraseña.equals(datosUsuario[1])) {
        autenticado = true;
        break;
    }
}
if (autenticado) {
    int[] notasAlberto = {8, 10, 9, 10, 10};
    Alumno alumno = new Alumno("Alberto", notasAlberto);
    System.out.println("Nota final de " + alumno.nombre + ": " + alumno.calcularNotaFinal());
    break;
} else {
    System.out.println("Nombre de usuario o contraseña incorrectos, te quedan " + (--intentos)
+ " intentos");
}
}
if (intentos == 0) {
    System.out.println("Has agotado el número de intentos permitidos");
}
}
}

```

23.7 Sobrecarga de constructores.

Podemos redefinir varios constructores con el mismo nombre.

23.8 Atributos y métodos de clase.

Tenemos que entender dos conceptos, los atributos y métodos de instancia, como hemos visto ahora y los atributos y métodos de clase. Para ello usamos Static

La diferencia esta en que los atributos y métodos de clase son exclusivos de la clase, y no es para cada objeto. Se usa para hacer el cálculo global de algún parámetro.

```

abstract class Empleado {
    private String nombre;

```



```

private double salario;
private static double totalSalarios;

public Empleado(String nombre, double salario) {
    this.nombre = nombre;
    this.salario = salario;
    totalSalarios += salario;
}

public String getNombre() {
    return nombre;
}

public double getSalario() {
    return salario;
}

public static double getTotalSalarios() {
    return totalSalarios;
}

abstract public void calcularPago();
}

```

La clase abstracta Empleado define los atributos nombre, salario, y un atributo estático totalSalarios. En el constructor, el valor del salario se suma a totalSalarios. Además, se define un método abstracto calcularPago que debe ser implementado por las clases hijas.

Luego, puedes crear una clase hija concreta que implemente el método calcularPago:

```

class EmpleadoFijo extends Empleado {
    public EmpleadoFijo(String nombre, double salario) {
        super(nombre, salario);
    }
}

```

```

@Override
public void calcularPago() {
    System.out.println("Pagando a " + getNombre() + ": " + getSalario());
}
}

```

En el main, puedes crear objetos de la clase EmpleadoFijo y acceder a su salario y al total de salarios:

```

public static void main(String[] args) {
    EmpleadoFijo empleado1 = new EmpleadoFijo("Juan", 1000);
    EmpleadoFijo empleado2 = new EmpleadoFijo("Pedro", 2000);

    empleado1.calcularPago();
    empleado2.calcularPago();

    System.out.println("Total de salarios: " + Empleado.getTotalSalarios());
}

```

El resultado sería:

```

Pagando a Juan: 1000.0
Pagando a Pedro: 2000.0
Total de salarios: 3000.0

```

Ejercicio

Realizar el mismo ejercicio, pero con notas y asignaturas, al menos 2 notas y que calcule la nota media.

```

abstract class Notas {
    private int nota1;
    private int nota2;
    private static float mediaNotas;
    private String asignatura1;
    private String asignatura2;

    public Notas(int nota1, int nota2, String asignatura1, String asignatura2) {

```

```

    this.nota1 = nota1;
    this.nota2 = nota2;
    this.asignatura1 = asignatura1;
    this.asignatura2 = asignatura2;
    this.mediaNotas += (nota1 + nota2) / 2.0f;
}

public int getNota1() {
    return nota1;
}

public int getNota2() {
    return nota2;
}

public static float getMediaNotas() {
    return mediaNotas;
}

public String getAsignatura1() {
    return asignatura1;
}

public String getAsignatura2() {
    return asignatura2;
}
}

class Alumno extends Notas {
    private String nombre;
    private String apellidos;

```

```

public Alumno(String nombre, String apellidos, int nota1, int nota2, String asignatura1, String
asignatura2) {
    super(nota1, nota2, asignatura1, asignatura2);
    this.nombre = nombre;
    this.apellidos = apellidos;
}

public String getNombre() {
    return nombre;
}

public String getApellidos() {
    return apellidos;
}
}

public class AlumnosMain {
    public static void main(String[] args) {
        Alumno alumno1 = new Alumno("Alberto", "Pérez", 10, 9, "Base de Datos", "Programación");
        Alumno alumno2 = new Alumno("Juan", "Gómez", 8, 8, "Entorno", "Sistemas");
        System.out.println("La media de las notas es: " + Notas.getMediaNotas());
    }
}

```

23.9 Interfaces

Las características son:

1. Contiene la cabecera de una serie de métodos, pero a modo de esquema. Por tanto, define el que y no el cómo.
2. A veces los programadores hacen interfaces que otros programadores desarrollan.
3. Usamos la palabra implements.

Ejemplo

Clase interfaz

```

public interface Operaciones {
    int sumar(int a, int b);
    int restar(int a, int b);
}

```

Clase Calculadora que implementa la interfaz

```
public class Calculadora implements Operaciones{

    @Override
    public int sumar(int a, int b) {
        // TODO Esbozo de método generado automáticamente
        return a+b;
    }

    @Override
    public int restar(int a, int b) {
        // TODO Esbozo de método generado automáticamente
        return a-b;
    }

}
```

Clase OperacionesMain

```
public class OperacionesMain {

    public static void main(String[] args) {
        // TODO Esbozo de método generado automáticamente
        Calculadora miCalculadora = new Calculadora();
        System.out.println(miCalculadora.sumar(5, 3));
        System.out.println(miCalculadora.restar(5, 3));
    }

}
```

Ejemplo

El mismo ejemplo anterior, pero con nota media y suma de nota

```
interface Notas {

    int calcularNotaMedia();

    int calcularSumaNotas();

}

class Alumno implements Notas {

    private int nota1;

    private int nota2;

    private int nota3;
```

```

public Alumno(int nota1, int nota2, int nota3) {
    this.nota1 = nota1;
    this.nota2 = nota2;
    this.nota3 = nota3;
}

public int calcularNotaMedia() {
    return (nota1 + nota2 + nota3) / 3;
}

public int calcularSumaNotas() {
    return nota1 + nota2 + nota3;
}
}

public class Main {
    public static void main(String[] args) {
        Alumno alumno1 = new Alumno(10, 8, 9);
        Alumno alumno2 = new Alumno(9, 7, 8);
        System.out.println("Nota media alumno 1: " + alumno1.calcularNotaMedia());
        System.out.println("Suma de notas alumno 1: " + alumno1.calcularSumaNotas());
        System.out.println("Nota media alumno 2: " + alumno2.calcularNotaMedia());
        System.out.println("Suma de notas alumno 2: " + alumno2.calcularSumaNotas());
    }
}

```

Realiza un ejercicio con salarios, nombre y complemento del salario de un 10%

```

interface CalculoSalario {
    double calcularSalario();
    double calcularComplemento();
}

```

```

class Empleado implements CalculoSalario {
    private String nombre;
    private double salario;

    Empleado(String nombre, double salario) {
        this.nombre = nombre;
        this.salario = salario;
    }

    @Override
    public double calcularSalario() {
        return this.salario;
    }

    @Override
    public double calcularComplemento() {
        return this.salario * 0.1;
    }
}

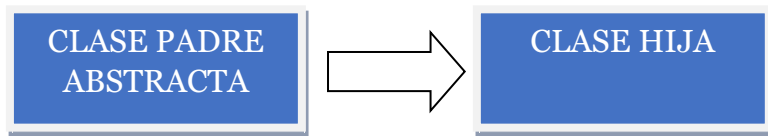
class Main {
    public static void main(String[] args) {
        Empleado empleado = new Empleado("Juan", 2000);
        System.out.println("Nombre: " + empleado.nombre);
        System.out.println("Salario: $" + empleado.calcularSalario());
        System.out.println("Complemento: $" + empleado.calcularComplemento());
        System.out.println("Salario Total: $" + (empleado.calcularSalario() +
empleado.calcularComplemento()));
    }
}

```

23.10 Diferencias entre clases abstractas e interfaces

Una clase abstracta puede heredar de una sola clase (abstracta o no) mientras que una interfaz puede extender varias interfaces de una misma vez. Una clase abstracta puede tener métodos

que sean abstractos o que no lo sean, mientras que las interfaces sólo y exclusivamente pueden definir métodos abstractos



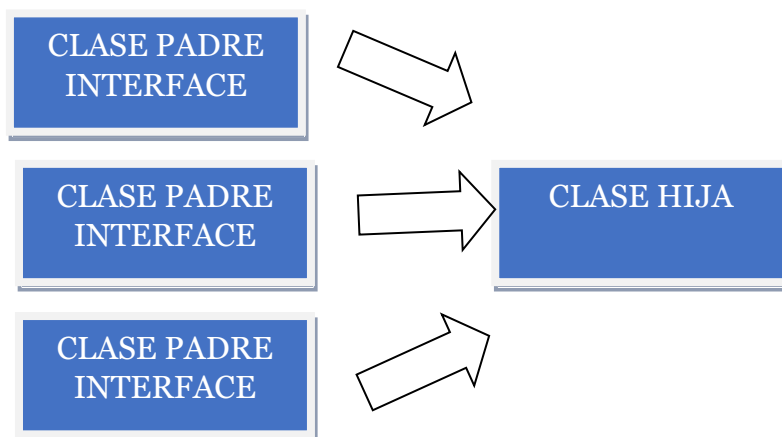
Métodos abstractos o no

Las interfaces tienen siempre métodos abstractos

Y una clase hija puede ser implementada por más de una interface

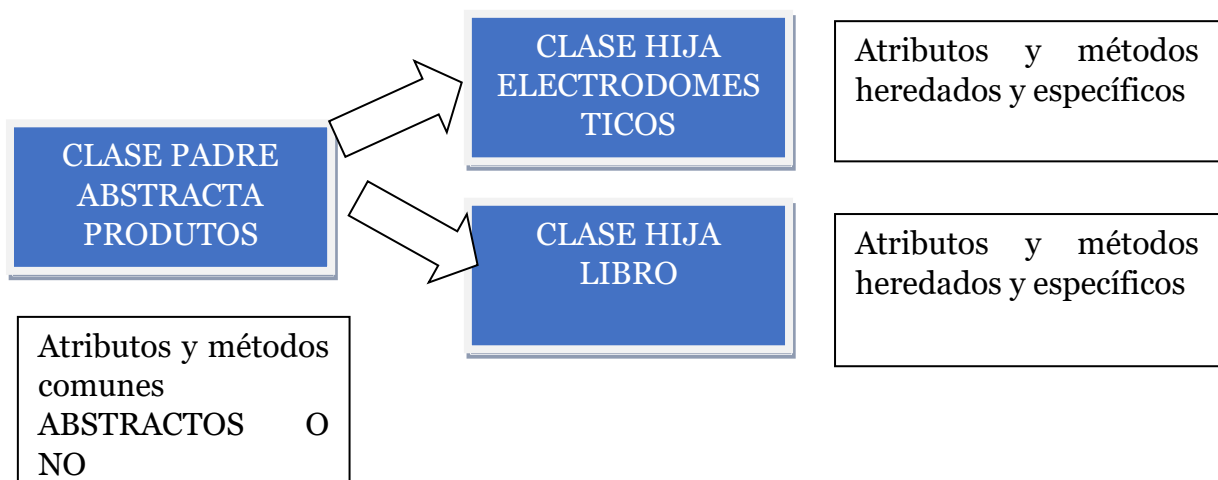
Se usan preferentemente en programación multihilo y también en interfaces gráficas.

Cuando se quiere que una serie de clases usen los mismos métodos con sus nombres y parámetros, usamos interfaces.

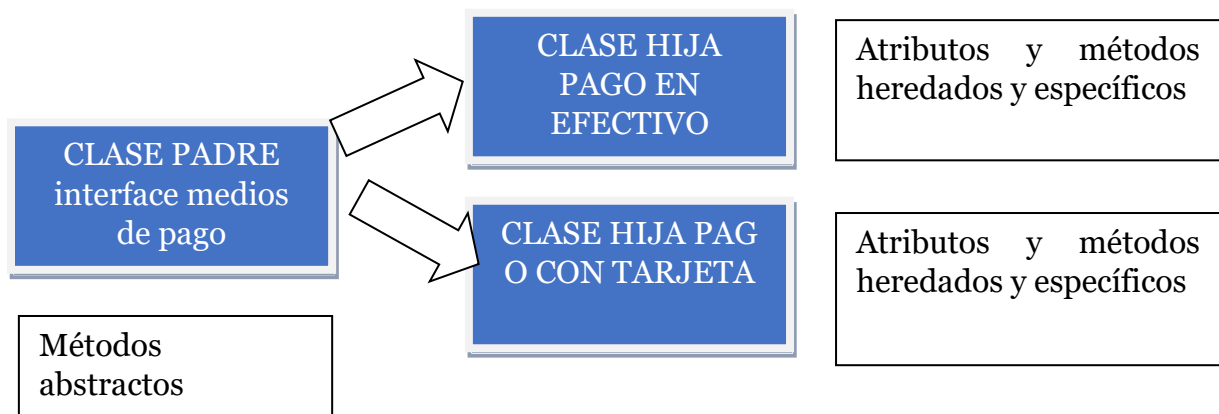


Solo métodos abstractos se definen en las interfaces,

Un ejemplo de uso de clases abstractas en la gestión de una tienda podría ser en productos. Podemos tener una clase abstracta Producto que defina atributos y métodos comunes a todos los productos, como el nombre, la descripción y el precio. Luego, podemos tener clases hijas específicas como Libro, Electrodoméstico, Ropa, etc. que hereden de la clase Producto y proporcionen información adicional específica para cada tipo de producto, como el autor de un libro o la marca de un electrodoméstico.



Un ejemplo de uso de interfaces en la gestión de una tienda podría ser el modelado de formas de pago. Podemos tener una interfaz Pagable que defina los métodos necesarios para realizar un pago, como calcular el total a pagar y confirmar el pago. Luego, podemos tener clases concretas como Tarjeta de Crédito, Transferencia Bancaria, Efectivo, etc. que implementen la interfaz Pagable y proporcionen su propia implementación para los métodos definidos en la interfaz. De esta manera, podemos permitir que los clientes paguen con diferentes métodos y tratarlos de manera uniforme en el código. Un uso frecuente de las interfaces es cuando trabajamos con interfaces gráficos o cuando usamos la programación multihilo.



Ahora vamos a ver el ejemplo primero, realizado con clases abstractas y con interfaces

Ejemplo con clases abstractas.

```

abstract class Producto {
    private String nombre;
    private String descripcion;
    private double precio;

    public Producto(String nombre, String descripcion,
double precio) {
        this.nombre = nombre;
        this.descripcion = descripcion;
        this.precio = precio;
    }

    public abstract void mostrarInformacion();

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
  
```

```

    }

    public String getDescripcion() {
        return descripcion;
    }

    public void setDescripcion(String descripcion) {
        this.descripcion = descripcion;
    }

    public double getPrecio() {
        return precio;
    }

    public void setPrecio(double precio) {
        this.precio = precio;
    }
}

class Libro extends Producto {
    private String autor;

    public Libro(String nombre, String descripcion, double
precio, String autor) {
        super(nombre, descripcion, precio);
        this.autor = autor;
    }

    @Override
    public void mostrarInformacion() {
        System.out.println("Nombre: " + getNombre());
        System.out.println("Descripcion: " +
getDescripcion());
        System.out.println("Precio: " + getPrecio());
        System.out.println("Autor: " + autor);
    }
}

class Electrodomestico extends Producto {
    private String marca;

```

```

    public Electrodomestico(String nombre, String
descripcion, double precio, String marca) {
        super(nombre, descripcion, precio);
        this.marca = marca;
    }

    @Override
    public void mostrarInformacion() {
        System.out.println("Nombre: " + getNombre());
        System.out.println("Descripcion: " +
getDescripcion());
        System.out.println("Precio: " + getPrecio());
        System.out.println("Marca: " + marca);
    }
}

```

```

import java.math.BigDecimal;

```

```

public class MainProducto {

    public static void main(String[] args) {
        Producto libro = new Libro("El señor de los
anillos", "Novela de aventura", 30.00, "Tolkien");
        Producto electrodomestico = new
Electrodomestico("Refrigerador", "Refrigerador de marca LG",
500, "LG");

        libro.mostrarInformacion();
        System.out.println();
        electrodomestico.mostrarInformacion();

    }

}

```

Ejemplo con interfaces

```

interface Productointerface {

    void mostrarInformacion();

}

```

```

class Librointerface implements Productointerface {

    private String nombre;
    private String autor;
    private Double precio;
    private String descripcion;

    public Librointerface(String nombre, String descripcion,
double precio, String autor) {

        this.autor = autor;
    }

    @Override
    public void mostrarInformacion() {
        System.out.println("Nombre: " + nombre);
        System.out.println("Descripcion: " + descripcion);
        System.out.println("Precio: " + precio);
        System.out.println("Autor: " + autor);
    }
}

```

```

lass Electrodomesticointerface implements Productointerface
{
    private String marca;
    private String nombre;
    private Double precio;
    private String descripcion;

    public Electrodomesticointerface(String nombre, String
descripcion, double precio, String marca) {

        this.marca = marca;
    }

    @Override
    public void mostrarInformacion() {
        System.out.println("Nombre: " + marca);
        System.out.println("Descripcion: " + descripcion);
        System.out.println("Precio: " + precio);
        System.out.println("Marca: " + marca);
    }
}

```

```

    }
}

public class MainProductoInterface {

    public static void main(String[] args) {
        Producto libro = new Libro("El señor de los
anillos", "Novela de aventura", 30.00, "Tolkien");
        Producto electrodomestico = new
Electrodomestico("Refrigerador", "Refrigerador de marca LG",
500, "LG");

        libro.mostrarInformacion();
        System.out.println();
        electrodomestico.mostrarInformacion();

    }

}

```

Ejemplo de pagos realizado en interfaces

```

interface Pagable {
    double calculateTotal();
    boolean confirmPayment();
}

class TarjetaDeCredito implements Pagable {
    @Override
    public double calculateTotal() {
        // Cálculo específico para Tarjeta de Crédito
        return 0;
    }

    @Override
    public boolean confirmPayment() {
        // Confirmación específica para Tarjeta de Crédito
        return true;
    }
}

```

```

class TransferenciaBancaria implements Pagable {
    @Override
    public double calculateTotal() {
        // Cálculo específico para Transferencia Bancaria
        return 0;
    }

    @Override
    public boolean confirmPayment() {
        // Confirmación específica para Transferencia Bancaria
        return true;
    }
}

class Efectivo implements Pagable {
    @Override
    public double calculateTotal() {
        // Cálculo específico para Efectivo
        return 0;
    }

    @Override
    public boolean confirmPayment() {
        // Confirmación específica para Efectivo
        return true;
    }
}

```

23.11 Arrays de objetos.

La clase **ArrayList en Java**, es una clase que permite almacenar datos en memoria de forma similar a los Arrays, con la ventaja de que el número de elementos que almacena, lo hace de forma dinámica, es decir, que no es necesario declarar su tamaño como pasa con los Arrays

CRUD (Create, Read, Update, Delete) **es un acrónimo para las maneras en las que se puede operar sobre información almacenada.** Es un nemónico para las cuatro funciones del almacenamiento persistente

USO DEL MÉTODO CRUD CON ARRAY DE OBJETOS

```
import java.util.Scanner;

class Empleado {
    private int id;
    private String nombre;
    private double salario;

    public Empleado(int id, String nombre, double salario) {
        this.id = id;
        this.nombre = nombre;
        this.salario = salario;
    }

    public int getId() {
        return id;
    }

    public String getNombre() {
        return nombre;
    }

    public double getSalario() {
        return salario;
    }
}

interface CRUD {
    void create(Empleado e);
    Empleado read(int id);
    void update(Empleado e);
    void delete(int id);
}

class EmpleadoDB implements CRUD {
```

```

private Empleado[] empleados = new Empleado[10];
private int contador = 0;

public void create(Empleado e) {
    empleados[contador++] = e;
}

public Empleado read(int id) {
    for (int i = 0; i < contador; i++) {
        if (empleados[i].getId() == id) {
            return empleados[i];
        }
    }
    return null;
}

public void update(Empleado e) {
    for (int i = 0; i < contador; i++) {
        if (empleados[i].getId() == e.getId()) {
            empleados[i].nombre = e.nombre;
            empleados[i].salario = e.salario;
            break;
        }
    }
}

public void delete(int id) {
    for (int i = 0; i < contador; i++) {
        if (empleados[i].getId() == id) {
            for (int j = i; j < contador - 1; j++) {
                empleados[j] = empleados[j + 1];
            }
            contador--;
            break;
        }
    }
}

```



```

    }
}

public class Main {
    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        EmpleadoDB db = new EmpleadoDB();

        db.create(new Empleado(1, "Alberto", 1000));
        db.create(new Empleado(2, "Juan", 2000));

        System.out.println("Lista de empleados:");
        for (int i = 0; i < db.contador; i++) {
            Empleado e = db.empleados[i];
            System.out.println("ID: " + e.getId() + " Nombre: " + e.getNombre() + " Salario: " +
e.getSalario());
        }

        System.out.println("\nActualizar salario:");
        System.out.print("ID: ");
        int id = scan.nextInt();

        Scanner scan = new Scanner(System.in);
        EmpleadoDB db = new EmpleadoDB();

        db.create(new Empleado(1, "Alberto", 1000));
        db.create(new Empleado(2, "Juan", 2000));

        System.out.println("Lista de empleados:");
        for (int i = 0; i < db.contador; i++) {
            Empleado e = db.empleados[i];
            System.out.println("ID: " + e.getId() + " Nombre: " + e.getNombre() + " Salario: " +
e.getSalario());
        }
    }
}

```

```

System.out.println("\nActualizar salario:");
System.out.print("ID: ");
int id = scan.nextInt();
System.out.print("Nuevo salario: ");
double salario = scan.nextDouble();
Empleado e = db.read(id);
if (e != null) {
    e.setSalario(salario);
    db.update(e);
    System.out.println("Empleado actualizado!");
} else {
    System.out.println("Empleado no encontrado");
}

System.out.println("\nLista de empleados:");
for (int i = 0; i < db.contador; i++) {
    e = db.empleados[i];
    System.out.println("ID: " + e.getId() + " Nombre: " + e.getNombre() + " Salario: " +
e.getSalario());
}

System.out.println("\nEliminar empleado:");
System.out.print("ID: ");
id = scan.nextInt();
db.delete(id);
System.out.println("Empleado eliminado!");

System.out.println("\nLista de empleados:");
for (int i = 0; i < db.contador; i++) {
    e = db.empleados[i];
    System.out.println("ID: " + e.getId() + " Nombre: " + e.getNombre() + " Salario: " +
e.getSalario());
}
scan.close();

```

24 COLECCIONES Y MÉTODOS

Una colección en Java es una estructura de datos que permite almacenar muchos valores del mismo tipo; por tanto, conceptualmente es prácticamente igual que un array.

(lista),
ArrayList .

(cola),
ArrayBlockingQueue

(conjunto),
HashSet

(pila),
Stack

Lista:

permite añadir, borrar, insertar, etc. Es mejor que usar un array, si queremos interactuar con la lista.

Cola

Es una estructura de datos que permite almacenar elementos en orden de llegada (First In, First Out, o FIFO).

Conjunto

Un conjunto en Java es una colección de elementos que, como el conjunto en matemáticas, no permite elementos duplicados dentro de ella y no tiene orden entre sus elementos

Pila

Es una estructura de datos que permite almacenar elementos y acceder a ellos en un orden específico, conocido como Last In, First Out (LIFO). Es decir, el último elemento que se agrega a la pila es el primero en salir.

¿Cuándo saber cuál emplear?

La elección de la estructura de datos adecuada en Java depende de las necesidades específicas del problema que se está resolviendo. A continuación, se presentan algunos casos de uso comunes para las estructuras de datos más comunes en Java:

ArrayList: se utiliza cuando se necesitan operaciones de búsqueda rápidas y se sabe de antemano el tamaño aproximado de la lista. El acceso a los elementos se realiza mediante índices y el rendimiento de las operaciones de inserción y eliminación es proporcional al tamaño de la lista.

Pila (Stack): se utiliza cuando se necesita un seguimiento de las operaciones que se han realizado y se requiere una reversión de las operaciones. Se utiliza para implementar algoritmos de búsqueda en profundidad, compiladores y para manejar excepciones en Java.

Conjunto (Set): se utiliza cuando se requiere una colección de elementos sin duplicados y no se necesita un orden específico de los elementos. Es útil para la eliminación de duplicados en una colección y para determinar si un elemento ya se encuentra en una colección. Las implementaciones HashSet y LinkedHashSet son comunes.

Cola (Queue): se utiliza cuando se necesita procesar elementos en orden de llegada (FIFO) y no es necesario acceder a elementos en el medio de la cola. Es útil para la implementación de sistemas de control de eventos y procesamiento de trabajos por lotes. Las implementaciones LinkedList y ArrayDeque son comunes.

Es importante tener en cuenta que, en muchos casos, la elección de la estructura de datos adecuada dependerá de las operaciones que se realizarán con mayor frecuencia, así como de la cantidad de elementos que se espera que contenga la colección. Además, también es importante considerar las limitaciones de memoria y rendimiento al elegir una estructura de datos para una tarea específica.

Ejemplos

ArrayList: Un ejemplo común de uso de ArrayList es para almacenar una lista de nombres de estudiantes. Si se sabe de antemano el número aproximado de estudiantes, se puede crear una instancia de ArrayList con ese tamaño inicial y agregar los nombres de los estudiantes a la lista. Luego, se pueden buscar los nombres de los estudiantes por índice para realizar operaciones como eliminar o actualizar nombres de estudiantes específicos.

Pila (Stack): Un ejemplo común de uso de la pila es para la implementación de un compilador. Durante el análisis sintáctico, el compilador utiliza una pila para realizar un seguimiento de los tokens y las reglas de la gramática que se han encontrado. Si se encuentra un error sintáctico, el compilador puede volver atrás y deshacer las operaciones de la pila para intentar encontrar un camino sintácticamente válido.

Conjunto (Set): Un ejemplo común de uso de Set es para almacenar una lista de direcciones de correo electrónico. Al utilizar un conjunto, se puede garantizar que no haya duplicados en la lista, lo que puede ser útil para enviar correos electrónicos masivos. Además, se puede utilizar el método "contains" para determinar si una dirección de correo electrónico ya está en la lista antes de agregarla.

Cola (Queue): Un ejemplo común de uso de la cola es para implementar un sistema de gestión de pedidos en un restaurante de comida rápida. Cuando se realiza un pedido, se agrega a la cola y se procesa en orden de llegada. Si se está procesando un pedido y se recibe otro, se agrega a la cola y se procesa después del primer pedido. De esta manera, se asegura que los pedidos se manejen en el orden en que se recibieron.

24.1 Interfaz Collection

Es la interfaz raíz de la lista, set o que. Hereda de la interfaz iterable, por esa razón podemos recorrer las colecciones usando un iterador. Todas pueden almacenar objetos duplicados, el caso de HasMap que no hereda de Collection no permite objetos duplicados.

Métodos.

- `int size()` Devuelve el número de elementos contenidos en la colección.
- `boolean isEmpty()` Devuelve `true` si no existen elementos en la colección.
- `void clear()` Vacía la colección.
- `boolean contains(Object elemento)` Retorna `true` en caso de que el elemento ya exista en la colección.
- `boolean containsAll(Collection<?> c2)` Retorna `true` en caso de que `c2` sea un subconjunto de la colección.
- `boolean add(E elemento)` Añade el elemento que se le pasa por argumento y devuelve `true` si la colección se ha modificado.
- `boolean addAll(Collection<? extends E> c2)` Añade a la colección todos los elementos de la colección `c2`. Retorna `true` si la colección original ha sido modificada.
- `boolean remove(Object elemento)` Elimina el elemento que se le pasa por argumento. Retorna `true` si el elemento ha podido eliminarse o `false` en caso contrario.
- `boolean removeAll(Collection<?> c2)` Elimina de la colección los elementos presentes en la colección `c2`, devolviendo `true` si la colección se ve modificada.
- `boolean retainAll(Collection<?> c2)` Elimina de la colección los elementos que no están presentes en la colección `c2`, devolviendo `true` si la colección se ve modificada.
- `Iterator<E> iterator()` Retorna un iterador sobre la colección de elementos tipo `E` para recorrerla en orden.
- `Object[] toArray()` Devuelve un `array` que contiene los elementos de la colección.

24.2 Interfaces set, map, list

En Java, las interfaces List, Map y Set son tres tipos de colecciones de datos que se pueden utilizar para almacenar y manipular elementos en un programa. Aquí hay algunos ejemplos de cuándo utilizar cada una de estas interfaces:

Interfaz	Para que sirve	Ejemplos	Clases
List:	La interfaz List se utiliza cuando se necesita una colección ordenada de elementos, en la que se pueden tener elementos duplicados.	• Para almacenar una lista de artículos en un carrito de compras en línea. • Para almacenar una lista de canciones en una lista de reproducción	Stack pila: En Java, una pila es una estructura de datos que sigue el principio de Last In First Out (LIFO), LinkedList: en Java es una clase que implementa la interfaz List y se utiliza para almacenar una colección de elementos en una lista enlazada ArrayList: es una clase que crea una matriz dinámica.
Map:	La interfaz Map se utiliza cuando se necesita almacenar elementos en una estructura de clave-valor, donde cada valor está asociado con una clave única	• Para almacenar los nombres de usuario y contraseñas en un sistema de inicio de sesión. • Para almacenar una lista de productos y sus precios en una tienda en línea. • Para almacenar una lista de nombres de personas y sus edades.	Un HashMap: en Java es una estructura de datos que permite almacenar pares clave-valor de forma eficiente. LinkedHashMap: es una implementación de Map que mantiene el orden de inserción de los elementos. Es decir, los elementos se almacenan en orden de inserción TreeMap: es una implementación de Map que mantiene los elementos ordenados según su clave. Es decir, los elementos se almacenan en orden natural
Set	La interfaz Set se utiliza cuando se necesita almacenar elementos únicos sin importar su orden	• Para almacenar una lista de correos electrónicos sin duplicados en una lista de suscriptores. • Para almacenar una lista de palabras clave únicas en una base de datos de búsqueda	HashSet: Los elementos en un HashSet no tienen un orden específico. LinkedHashSet: si los ordena según el orden que haya sido insertado. TreeSet: lo hace de manera ascendente.
Queue	En Java, la interfaz Queue define una estructura de datos de tipo cola, que sigue el principio "primero en entrar, primero en salir" (FIFO, por sus siglas en	• Por ejemplo, cuando se procesan eventos de entrada de usuario en una aplicación, se pueden agregar a una cola y procesarlos en el orden en que se recibieron. • Por ejemplo, en una aplicación de procesamiento por	Queue: una cola es una estructura de datos que sigue el principio de First In First Out (FIFO), PriorityQueue: es una cola de prioridad, es decir, los elementos se agregan y eliminan en función de su prioridad, ArrayQueue: es una cola

	inglés	lotes, se pueden agregar tareas a una cola y procesarlas en el orden en que se recibieron.	implementada utilizando un array. Los elementos en una
--	--------	--	--

24.3 Interfaz Set: HashSet, LinkedHashSet, TreeSet

La interfaz Set en Java es una colección que no permite elementos duplicados. Esta interfaz extiende la interfaz Collection y define el comportamiento de un conjunto (conjunto matemático). Los elementos en un conjunto no tienen un orden específico.

Hay tres clases que implementan la interfaz Set en Java: HashSet, LinkedHashSet y TreeSet. A continuación, se muestra un ejemplo de cómo se pueden usar estas clases:

HashSet: **HashSet** es una implementación de la interfaz Set que utiliza una tabla hash para almacenar los elementos.

Orden:

HashSet: Los elementos en un HashSet no tienen un orden específico.

LinkedHashSet si los ordena según el orden que haya sido insertado.

TreeSet lo hace de manera ascendente.

HashSet;

```
import java.util.HashSet;

public class HashSetExample {
    public static void main(String[] args) {
        HashSet<String> hashSet = new HashSet<>();
        hashSet.add("superalbertron");
        hashSet.add("javitron");
        hashSet.add("pablitron");
        hashSet.add("superalbertron"); // Duplicado, no se agrega
    }
}
```

```
        System.out.println(hashSet);
    }
}
```

LinkedHashSet

```
import java.util.LinkedHashSet;

public class LinkedHashSetEjemplo {

    public static void main(String[] args) {
        LinkedHashSet<String> linkedHashSet = new LinkedHashSet<>();
        linkedHashSet.add("superalbertron");
        linkedHashSet.add("javitron");
        linkedHashSet.add("pablitron");
        System.out.println(linkedHashSet);
    }
}
```

TreeSet:

TreeSet es una implementación de la interfaz Set que utiliza un árbol para almacenar los elementos. Los elementos en un TreeSet se almacenan en orden ascendente o descendente según el orden natural de los elementos o según un comparador proporcionado.

TreeSet

```
import java.util.TreeSet;

public class TreeSetExample {
    public static void main(String[] args) {
```



```
TreeSet<String> treeSet = new TreeSet<>();
treeSet.add("superalbertron");
treeSet.add("javitron");
treeSet.add("pablitron");
System.out.println(treeSet);
}
}
```

24.4 Interfaz list, ArrayList, Stack(pila), LinkedList

Tenemos los siguientes tipos:

- **Stack pila:** En Java, una pila es una estructura de datos que sigue el principio de Last In First Out (LIFO), lo que significa que el último elemento que se inserta en la pila es el primero en ser eliminado.
- **LinkedList** en Java es una clase que implementa la interfaz List y se utiliza para almacenar una colección de elementos en una lista enlazada
- **Arraylist:** es una clase que crea una matriz dinámica.

Arraylist

En Java, ArrayList es una clase que implementa la interfaz List y proporciona una implementación de una matriz dinámica que puede crecer o encogerse según sea necesario durante la ejecución del programa.

Las operaciones más comunes que se pueden realizar con un objeto de la clase

ArrayList son las siguientes:

add(elemento)

Añade un elemento al final de la lista.

add(indice, elemento)

Inserta un elemento en una posición determinada, desplazando el resto de elementos hacia la derecha.

clear()

Elimina todos los elementos pero no borra la lista.

contains(elemento)

Devuelve true si la lista contiene el elemento que se especifica y false en caso contrario.

get(indice)

Devuelve el elemento de la posición que se indica entre paréntesis.

indexOf(elemento)

Devuelve la posición de la primera ocurrencia del elemento que se indica entre paréntesis.

isEmpty()

Devuelve true si la lista está vacía y false en caso de tener algún elemento.

remove(indice)

Elimina el elemento que se encuentra en una posición determinada.

remove(elemento)

Elimina la primera ocurrencia de un elemento.

removeIf(filtro)

Elimina los elementos que cumplen una determinada condición.

set(indice, elemento)

Machaca el elemento que se encuentra en una determinada posición con el elemento que se pasa como parámetro.

size()

Devuelve el tamaño (número de elementos) de la lista.

toArray()

Devuelve un *array* con todos y cada uno de los elementos que contiene la lista.

Ejemplo

```
package arraylist;

import java.util.ArrayList;
/**
 * Ejemplo de uso de la clase ArrayList
 *
 * @author Alberto Ruiz
 */
public class Agregar {
    public static void main(String[] args) {

        ArrayList<String> asigna = new ArrayList<String>();

        System.out.println("Nº de elementos: " + asigna.size());

        asigna.add("Entorno");
        asigna.add("Programación");
        asigna.add("Base de datos");
        System.out.println("Nº de elementos: " + asigna.size());
        asigna.add("Sistemas");
        System.out.println("Nº de elementos: " + asigna.size());
        System.out.println("El elemento que hay en la posición 0 es " +
asigna.get(0));
        System.out.println("El elemento que hay en la posición 3 es " +
asigna.get(3));

    }
}

package arraylist;
```

```

import java.util.ArrayList;
/**
 * Ejemplo de uso de la clase ArrayList
 *
 * @author Alberto Ruiz
 */
public class Eliminar {
    public static void main(String[] args) {

        ArrayList<String> asigna = new ArrayList<String>();

        asigna.add("Entorno");
        asigna.add("Programación");
        asigna.add("Base de datos");
        asigna.add("Sistemas");
        asigna.add("matematicas");
        System.out.println("Nº de elementos: " + asigna.size());
        System.out.println(asigna);

        //definimos una función lambda con un nombre que lo que hace es
recorrer el arraylist
        asigna.removeIf(asignatura -> asignatura.equals("matematicas"));

        //Muestra todos los datos

        System.out.println("Nº de elementos: " + asigna.size());
        System.out.println(asigna);

    }
}

```

```

package arraylist;

```

```

import java.util.ArrayList;
/**
 * Ejemplo de uso de la clase ArrayList
 *
 * @author Alberto Ruiz
 */
public class Modificar {
    public static void main(String[] args) {
        String buscar="Word";

        ArrayList<String> asigna = new ArrayList<String>();
        asigna.add("Entorno");
        asigna.add("Programación");
        asigna.add("Base de datos");
        asigna.add("Sistemas");
        asigna.add("Word");
        System.out.println(asigna);
    }
}

```

```

        int position=asigna.indexOf(buscar);
        System.out.println(" la posicion de Word es "+position);
        asigna.set(position, "Marcas");
        System.out.println(asigna);
    }
}

package arraylist;

import java.util.ArrayList;
/**
 * Ejemplo de uso de la clase ArrayList
 *
 * @author Alberto Ruiz
 */
public class Mostrar {
    public static void main(String[] args) {

        ArrayList<String> asigna = new ArrayList<String>();
        asigna.add("Entorno");
        asigna.add("Programación");
        asigna.add("Base de datos");
        asigna.add("Sistemas");

        //Muestra todos los datos
        for (String a:asigna) {
            System.out.println("-"+ a);
        }
    }
}

```

Stack(pila)

En Java, una pila es una estructura de datos que sigue el principio de Last In First Out (LIFO), lo que significa que el último elemento que se inserta en la pila es el primero en ser eliminado.

Cuando emplear una pila en un programa

1. Procesamiento de expresiones matemáticas: Las pilas son muy útiles para procesar expresiones matemáticas en las que se necesite mantener un seguimiento de los operadores y operandos. Por ejemplo, para evaluar una expresión matemática en notación posfija, se puede utilizar una pila para almacenar los operandos y realizar las operaciones con los operadores.
2. Navegación en aplicaciones web: Las pilas pueden ser utilizadas para almacenar las páginas visitadas en una aplicación web, permitiendo al usuario navegar hacia atrás y adelante entre ellas.
3. Implementación de historiales: Las pilas pueden ser utilizadas para implementar historiales en diversas aplicaciones. Por ejemplo, para almacenar las acciones realizadas por un usuario en una aplicación y permitirle deshacerlas en orden inverso.
4. Gestión de llamadas a funciones: Las pilas son muy útiles para gestionar llamadas a funciones en la pila de ejecución de un programa. Cada vez que se llama a una función,

se puede insertar su dirección de retorno en la pila y, cuando la función termina, se puede utilizar esa dirección para volver al punto de origen.

5. Validación de estructuras: Las pilas pueden ser utilizadas para validar estructuras de datos como paréntesis, corchetes y llaves en un lenguaje de programación. Por ejemplo, se puede utilizar una pila para verificar si una expresión matemática está bien formada y tiene una sintaxis válida.

En Java, la clase Stack se puede utilizar para implementar una pila. Aquí hay un ejemplo de cómo crear y utilizar una pila en Java utilizando la clase Stack:

```
import java.util.Stack;

public class PilaExample {

    public static void main(String[] args) {

        // crear una pila

        Stack<String> pila = new Stack<>();

        // agregar elementos a la pila

        pila.push("Elemento 1");

        pila.push("Elemento 2");

        pila.push("Elemento 3");

        // obtener el elemento superior de la pila

        String elementoSuperior = pila.peek();

        System.out.println("El elemento superior es: " + elementoSuperior);

        // eliminar un elemento de la pila

        String elementoEliminado = pila.pop();

        System.out.println("El elemento eliminado es: " + elementoEliminado);

        // mostrar todos los elementos de la pila

        while (!pila.empty()) {

            String elemento = pila.pop();

            System.out.println("Elemento: " + elemento);

        }

    }

}
```

```
}  
  
}
```

En este ejemplo, primero creamos una pila utilizando la clase Stack. Luego, agregamos algunos elementos a la pila utilizando el método push(). Después, utilizamos el método peek() para obtener el elemento superior de la pila sin eliminarlo, y el método pop() para eliminar el elemento superior y obtener su valor.

Finalmente, utilizamos un bucle while para mostrar todos los elementos de la pila. El método empty() se utiliza para verificar si la pila está vacía o no.

LinkedList

LinkedList en Java es una clase que implementa la interfaz List y se utiliza para almacenar una colección de elementos en una lista enlazada. Cada elemento en la lista contiene un enlace al siguiente elemento, lo que permite un acceso rápido a los elementos adyacentes.

Cuando usar en un programa linkedlist

- **Insertar y eliminar elementos con frecuencia:** La LinkedList es muy útil cuando se necesita insertar o eliminar elementos con frecuencia, especialmente cuando se trata de elementos que no están en la posición final de la lista. Esto se debe a que la inserción y eliminación de elementos en una LinkedList es más eficiente que en un ArrayList.
- **Trabajar con listas grandes:** Si tienes una lista grande y necesitas insertar o eliminar elementos con frecuencia, es probable que LinkedList sea una opción más eficiente que ArrayList.
- **Iteración secuencial de los elementos:** La LinkedList es una buena opción si solo necesitas iterar secuencialmente a través de los elementos de la lista, ya que el acceso a cada elemento es constante y no depende de su posición en la lista.
- **Ordenación de la lista:** La LinkedList es adecuada para ordenar listas de elementos, ya que proporciona un método "sort"

Métodos

1. **add():** Este método se utiliza para agregar un elemento al final de la lista.
2. **addFirst():** Este método se utiliza para agregar un elemento al inicio de la lista.
3. **addLast():** Este método se utiliza para agregar un elemento al final de la lista.
4. **remove():** Este método se utiliza para eliminar un elemento de la lista.
5. **removeFirst():** Este método se utiliza para eliminar el primer elemento de la lista.
6. **removeLast():** Este método se utiliza para eliminar el último elemento de la lista.
7. **get():** Este método se utiliza para acceder a un elemento de la lista.
8. **size():** Este método se utiliza para obtener el tamaño de la lista.

9. **clear()**: Este método se utiliza para eliminar todos los elementos de la lista.
10. **isEmpty()**: Este método se utiliza para verificar si la lista está vacía o no.
11. **contains()**: Este método se utiliza para verificar si un elemento está presente en la lista.
12. **indexOf()**: Este método se utiliza para obtener el índice de la primera ocurrencia de un elemento en la lista.
13. **lastIndexOf()**: Este método se utiliza para obtener el índice de la última ocurrencia de un elemento en la lista.

```
import java.util.LinkedList;

public class LinkedListExample {
    public static void main(String[] args) {
        LinkedList<String> linkedList = new LinkedList<>();
        linkedList.add("elemento1");
        linkedList.add("elemento2");
        linkedList.add("elemento3");

        System.out.println("LinkedList: " + linkedList);

        // Agrega un elemento al inicio de la lista
        linkedList.addFirst("inicio");
        System.out.println("LinkedList después de agregar al inicio: " +
linkedList);

        // Agrega un elemento al final de la lista
        linkedList.addLast("fin");
        System.out.println("LinkedList después de agregar al final: " +
linkedList);

        // Accede al primer y último elemento de la lista
        String firstElement = linkedList.getFirst();
        String lastElement = linkedList.getLast();
        System.out.println("Primer elemento: " + firstElement);
        System.out.println("Último elemento: " + lastElement);

        // Remueve el primer y último elemento de la lista
        linkedList.removeFirst();
        linkedList.removeLast();
        System.out.println("LinkedList después de remover el primer y último
elemento: " + linkedList);

        // Remueve el elemento en la posición 1
        linkedList.remove(1);
        System.out.println("LinkedList después de remover el elemento en la
posición 1: " + linkedList);
    }
}
```

En este ejemplo, se crea una instancia de `LinkedList` y se agregan algunos elementos a la lista utilizando el método `add()`. Luego, se muestra la lista completa utilizando el método `toString()`.

A continuación, se agregan elementos al inicio y al final de la lista utilizando los métodos `addFirst()` y `addLast()`, respectivamente.

Luego, se accede al primer y último elemento de la lista utilizando los métodos `getFirst()` y `getLast()`, respectivamente.

Después, se remueven el primer y último elemento de la lista utilizando los métodos `removeFirst()` y `removeLast()`, respectivamente.

Finalmente, se remueve el elemento en la posición 1 de la lista utilizando el método `remove()`.

24.5 Interfaz Queue(cola): PriorityQueue, ArrayQueue

En Java, una cola es una estructura de datos que sigue el principio de First In First Out (FIFO), lo que significa que el primer elemento que se inserta en la cola es el primero en ser eliminado.

Cuando emplear una cola

1. Procesamiento de tareas en segundo plano: Una cola puede usarse para procesar tareas en segundo plano en un orden específico. Por ejemplo, se pueden agregar tareas a la cola y se procesan en el orden en que se agregaron. Esto es útil cuando se necesita procesar tareas en segundo plano de manera ordenada y no se puede permitir que se ejecuten al mismo tiempo.
2. Procesamiento de eventos en tiempo real: Una cola también puede ser útil para procesar eventos en tiempo real en un orden específico. Por ejemplo, se pueden agregar eventos a la cola y se procesan en el orden en que se agregaron. Esto es útil cuando se necesita procesar eventos en tiempo real y se debe mantener el orden en que se recibieron.
3. Almacenamiento de mensajes de correo electrónico: Una cola puede usarse para almacenar mensajes de correo electrónico en un orden específico. Por ejemplo, se pueden agregar mensajes a la cola y se entregan en el orden en que se agregaron. Esto es útil cuando se necesita enviar mensajes de correo electrónico en un orden específico y no se puede permitir que se envíen al mismo tiempo.
4. Gestión de procesos: Una cola también puede ser útil para la gestión de procesos. Por ejemplo, se pueden agregar procesos a la cola y se ejecutan en el orden en que se agregaron. Esto es útil cuando se necesita ejecutar procesos en un orden específico y no se puede permitir que se ejecuten al mismo tiempo.

Métodos

1. `add(E elemento)`: Agrega un elemento a la cola, si es posible hacerlo inmediatamente, de lo contrario arroja una excepción.
2. `offer(E elemento)`: Agrega un elemento a la cola, si es posible hacerlo inmediatamente, y devuelve `true`. Si la cola está llena, no agrega el elemento y devuelve `false`.
3. `remove()`: Elimina y devuelve el elemento en la cabeza de la cola. Si la cola está vacía, arroja una excepción.

4. `poll()`: Elimina y devuelve el elemento en la cabeza de la cola. Si la cola está vacía, devuelve `null`.
5. `element()`: Devuelve el elemento en la cabeza de la cola sin eliminarlo. Si la cola está vacía, arroja una excepción.
6. `peek()`: Devuelve el elemento en la cabeza de la cola sin eliminarlo. Si la cola está vacía, devuelve `null`.

En Java, la clase **Queue** se puede utilizar para implementar una cola. Aquí hay un ejemplo de cómo crear y utilizar una cola en Java utilizando la clase Queue:

```
import java.util.LinkedList;

import java.util.Queue;

public class ColaExample {

    public static void main(String[] args) {

        // crear una cola

        Queue<String> cola = new LinkedList<>();

        // agregar elementos a la cola

        cola.add("Elemento 1");
        cola.add("Elemento 2");
        cola.add("Elemento 3");

        // obtener el primer elemento de la cola

        String primerElemento = cola.peek();

        System.out.println("El primer elemento es: " + primerElemento);

        // eliminar un elemento de la cola

        String elementoEliminado = cola.remove();

        System.out.println("El elemento eliminado es: " + elementoEliminado);
```

```

// mostrar todos los elementos de la cola

while (!cola.isEmpty()) {

    String elemento = cola.remove();

    System.out.println("Elemento: " + elemento);

}

}

```

En este ejemplo, primero creamos una cola utilizando la clase `Queue`. Luego, agregamos algunos elementos a la cola utilizando el método `add()`. Después, utilizamos el método `peek()` para obtener el primer elemento de la cola sin eliminarlo, y el método `remove()` para eliminar el primer elemento y obtener su valor.

Finalmente, utilizamos un bucle `while` para mostrar todos los elementos de la cola. El método `isEmpty()` se utiliza para verificar si la cola está vacía o no.

PriorityQueue.

`PriorityQueue` es una cola de prioridad, es decir, los elementos se agregan y eliminan en función de su prioridad, que se determina por el orden natural de los elementos o por un comparador proporcionado. Los elementos en una `PriorityQueue` se almacenan en orden ascendente o descendente según su prioridad.

Aquí hay un ejemplo de cómo se puede usar `PriorityQueue` en Java:

```

import java.util.PriorityQueue;

public class PriorityQueueExample {
    public static void main(String[] args) {
        PriorityQueue<Integer> priorityQueue = new PriorityQueue<>();
        priorityQueue.add(3);
        priorityQueue.add(1);
        priorityQueue.add(2);

        System.out.println("PriorityQueue: " + priorityQueue);

        int firstElement = priorityQueue.poll();
        System.out.println("Primer elemento: " + firstElement);
    }
}

```

```

        System.out.println("PriorityQueue después de remover el primer
elemento: " + priorityQueue);
    }
}

```

En este ejemplo, se crea una instancia de `PriorityQueue` y se agregan algunos elementos a la cola utilizando el método `add()`. Luego, se muestra la cola completa utilizando el método `toString()`.

Después, se remueve el primer elemento de la cola utilizando el método `poll()` y se muestra el elemento removido.

Finalmente, se muestra la cola actualizada después de remover el primer elemento.

ArrayQueue

`ArrayQueue` es una cola implementada utilizando un array. Los elementos en una `ArrayQueue` se agregan al final del arreglo y se eliminan del principio del arreglo.

Aquí hay un ejemplo de cómo se puede usar `ArrayQueue` en Java:

```

import java.util.ArrayDeque;
import java.util.Queue;

public class ArrayQueueExample {
    public static void main(String[] args) {
        Queue<Integer> arrayQueue = new ArrayDeque<>();
        arrayQueue.add(1);
        arrayQueue.add(2);
        arrayQueue.add(3);

        System.out.println("ArrayQueue: " + arrayQueue);

        int firstElement = arrayQueue.poll();
        System.out.println("Primer elemento: " + firstElement);

        System.out.println("ArrayQueue después de remover el primer
elemento: " + arrayQueue);
    }
}

```

En este ejemplo, se crea una instancia de `ArrayDeque` y se agregan algunos elementos a la cola utilizando el método `add()`. Luego, se muestra la cola completa utilizando el método `toString()`.

Después, se remueve el primer elemento de la cola utilizando el método `poll()` y se muestra el elemento removido.

Finalmente, se muestra la cola actualizada después de remover el primer elemento.

24.6 Interfaz Map: HashMap, LinkedHashMap, TreeMap

Los métodos son los siguientes:

1. `put(K clave, V valor)`: Asocia el valor especificado con la clave especificada en este mapa. Si la clave ya estaba asociada a algún valor, entonces el valor anterior es reemplazado.
2. `get(Object clave)`: Devuelve el valor asociado con la clave especificada, o `null` si este mapa no contiene ninguna asignación para la clave.
3. `remove(Object clave)`: Elimina la asignación para la clave especificada de este mapa, si está presente.
4. `containsKey(Object clave)`: Devuelve `true` si este mapa contiene una asignación para la clave especificada.
5. `keySet()`: Devuelve un conjunto de todas las claves contenidas en este mapa.
6. `values()`: Devuelve una colección de todos los valores contenidos en este mapa.
7. `entrySet()`: Devuelve un conjunto de las entradas (pares clave-valor) contenidas en este mapa.

La interfaz `Map` es una interfaz en Java que define una colección de pares de clave-valor únicos. Cada entrada en un mapa contiene una clave única y un valor asociado a esa clave. La clave se utiliza para buscar y acceder al valor asociado en el mapa.

La interfaz `Map` proporciona métodos para agregar, eliminar y buscar entradas en el mapa, así como para obtener información sobre el tamaño del mapa y sus claves y valores. La interfaz `Map` también proporciona métodos para iterar sobre las entradas del mapa, que se pueden usar para realizar operaciones en todas las entradas del mapa.

Algunas implementaciones de la interfaz `Map` en Java incluyen `HashMap`, `TreeMap`, `LinkedHashMap` y `ConcurrentHashMap`. Cada implementación tiene diferentes propiedades y características, como el orden de las entradas en el mapa, la eficiencia en la búsqueda y la capacidad de manejar múltiples hilos de ejecución.

Aquí hay algunos ejemplos de cómo se puede usar la interfaz `Map` en Java:

```
java
import java.util.HashMap;
import java.util.Map;
```

La interfaz `Map` es una interfaz en Java que define una colección de pares de clave-valor únicos. Cada entrada en un mapa contiene una clave única y un valor asociado a esa clave. La clave se utiliza para buscar y acceder al valor asociado en el mapa.

La interfaz `Map` proporciona métodos para agregar, eliminar y buscar entradas en el mapa, así como para obtener información sobre el tamaño del mapa y sus claves y valores. La interfaz `Map` también proporciona métodos para iterar sobre las entradas del mapa, que se pueden usar para realizar operaciones en todas las entradas del mapa.

Algunas implementaciones de la interfaz `Map` en Java incluyen `HashMap`, `TreeMap`, `LinkedHashMap` y `ConcurrentHashMap`. Cada implementación tiene diferentes propiedades y características, como el orden de las entradas en el mapa, la eficiencia en la búsqueda y la capacidad de manejar múltiples hilos de ejecución.

Aquí hay algunos ejemplos de cómo se puede usar la interfaz `Map` en Java:

```
java
import java.util.HashMap;
import java.util.Map;
```

Cuando usar en un programa `HashMap`, `TreeMap` y `LinkedHashMap`

1. **HashMap:** Esta implementación es muy eficiente para realizar operaciones de búsqueda, inserción y eliminación en una gran cantidad de datos. Es recomendable usar `HashMap` si la aplicación necesita acceder a los datos de forma rápida y no se requiere un orden específico para los elementos.
2. **TreeMap:** A diferencia de `HashMap`, `TreeMap` ordena sus elementos según una clave natural o utilizando un comparador personalizado. Es recomendable usar `TreeMap` si se necesita una estructura de datos ordenada según alguna condición específica, como una clasificación alfabética de las claves. También es útil si se desea buscar elementos utilizando rangos de claves.
3. **LinkedHashMap:** Esta implementación es similar a `HashMap`, pero mantiene el orden de inserción de los elementos en la lista. Es recomendable usar `LinkedHashMap` si se necesita un orden específico para los elementos, pero también se requiere una operación de búsqueda rápida.

Un **HashMap** en Java es una estructura de datos que permite almacenar pares clave-valor de forma eficiente. En otras palabras, es una colección que permite asociar un valor a una clave, lo que facilita la búsqueda y recuperación de elementos.

Aquí te dejo un ejemplo de cómo usar un `HashMap` en Java con una lista de correos electrónicos:

```
import java.util.HashMap;
```

```

public class HashMapExample {

    public static void main(String[] args) {

        HashMap<String, String> emails = new HashMap<>();

        // agregar correos electrónicos al HashMap

        emails.put("Juan Perez", "juan.perez@example.com");

        emails.put("Maria Rodriguez", "maria.rodriguez@example.com");

        emails.put("Pedro Gomez", "pedro.gomez@example.com");


        // obtener un correo electrónico por su clave

        String email = emails.get("Maria Rodriguez");

        System.out.println("El correo electrónico de Maria Rodriguez es " + email);


        // recorrer todos los elementos del HashMap

        for (String key : emails.keySet()) {

            String value = emails.get(key);

            System.out.println(key + " -> " + value);

        }

    }

}

```

En este ejemplo, creamos un HashMap llamado "emails" que tiene cadenas de texto como clave y valor. Luego, agregamos algunos correos electrónicos al HashMap utilizando el método put(). Después, obtenemos el correo electrónico de María Rodríguez utilizando el método get() y lo imprimimos en la consola.

Finalmente, recorreremos todos los elementos del HashMap utilizando un bucle for-each y mostramos cada clave y valor en la consola.

LinkedHashMap.

`LinkedHashMap` es una implementación de `Map` que mantiene el orden de inserción de los elementos. Es decir, los elementos se almacenan en orden de inserción y se pueden iterar en ese mismo orden. Cada entrada en la mapa contiene un par de clave-valor.

Aquí hay un ejemplo de cómo se puede usar `LinkedHashMap` en Java:

```
import java.util.LinkedHashMap;

public class LinkedHashMapEjemplo {
    public static void main(String[] args) {
        LinkedHashMap<String, Integer> linkedHashMap = new LinkedHashMap<>();
        linkedHashMap.put("Java", 1);
        linkedHashMap.put("Python", 2);
        linkedHashMap.put("JavaScript", 3);

        System.out.println("LinkedHashMap: " + linkedHashMap);

        int value = linkedHashMap.get("Java");
        System.out.println("Valor asociado a la clave 'Java': " + value);
    }
}
```

En este ejemplo, se crea una instancia de `LinkedHashMap` y se agregan algunos elementos a la mapa utilizando el método `put()`. Luego, se muestra la mapa completa utilizando el método `toString()`.

Después, se obtiene el valor asociado a la clave "dos" utilizando el método `get()` y se muestra el valor.

Ejemplo de ordenación

```
import java.util.LinkedHashMap;

import java.util.List;

import java.util.Map;

import java.util.stream.Collectors;

public class LinkedHashMapEjemplo {

    public static void main(String[] args) {

        LinkedHashMap<String, Integer> linkedHashMap = new LinkedHashMap<>();

        linkedHashMap.put("Java", 2);

        linkedHashMap.put("Python", 1);
```

```

linkedHashMap.put("JavaScript", 3);

System.out.println("LinkedHashMap sin ordenar: " + linkedHashMap);

List<Map.Entry<String, Integer>> list = linkedHashMap.entrySet().stream()
    .sorted(Map.Entry.comparingByValue())
    .collect(Collectors.toList());

LinkedHashMap<String, Integer> linkedHashMapOrdenado = new LinkedHashMap<>();
for (Map.Entry<String, Integer> entry : list) {
    linkedHashMapOrdenado.put(entry.getKey(), entry.getValue());
}

System.out.println("LinkedHashMap ordenado por valor: " + linkedHashMapOrdenado);
}
}

```

TreeMap

`TreeMap` es una implementación de `Map` que mantiene los elementos ordenados según su clave. Es decir, los elementos se almacenan en orden natural (o definido por el comparador proporcionado) de las claves. Cada entrada en el mapa contiene un par de clave-valor.

Aquí hay un ejemplo de cómo se puede usar `TreeMap` en Java:

```

import java.util.TreeMap;

import java.util.Map;

public class TreeMapExample {

    public static void main(String[] args) {

        Map<String, Integer> treeMap = new TreeMap<>();
    }
}

```



```

treeMap.put("tres", 3);

treeMap.put("uno", 1);

treeMap.put("dos", 2);


System.out.println("TreeMap: " + treeMap);


int value = treeMap.get("dos");

System.out.println("Valor asociado a la clave 'dos': " + value);

}

}

```

25 EXCEPCIONES

Existen otros errores que se producen en tiempo de ejecución a los que llamaremos

En Java, el bloque `try-catch` se utiliza para manejar excepciones que pueden ocurrir durante la ejecución de un programa. Un bloque `try` define un ámbito en el que se pueden producir excepciones y en el que se puede manejar dichas excepciones de manera controlada, mientras que el bloque `catch` se utiliza para manejar las excepciones que se han producido dentro del bloque `try`

El Bloque try catch finally

```

try {
    Instrucciones que se pretenden ejecutar
    (si se produce una excepción puede que
    no se ejecuten todas ellas).
} catch {
    Instrucciones que se van a ejecutar
    cuando se produce una excepción.
} finally {
    Instrucciones que se van a ejecutar tanto
    si se producen excepciones como si no
    se producen.
}

```

Nota: puedes utilizar varios catch.

```

public class EjemploExcepciones02 {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.println("Este programa calcula la media de dos números");
        try {
            System.out.print("Introduzca el primer número: ");
            double numero1 = Double.parseDouble(s.nextLine());

```

```

System.out.print("Introduzca el segundo número: ");
double numero2 = Double.parseDouble(s.nextLine());
System.out.println("La media es " + (numero1 + numero2) / 2);
} catch (Exception e) {
System.out.print("No se puede calcular la media. ");
System.out.println("Los datos introducidos no son correctos.");
} finally {
System.out.println("Gracias por utilizar este programa ¡hasta la próxima!");
}
}
}

```

Ejemplo 2

```

import java.util.Scanner;
public class EjemploExcepciones02v2 {
public static void main(String[] args) {
Scanner s = new Scanner(System.in);
System.out.println("Este programa calcula la media de dos números");
try {
System.out.print("Introduzca el primer número: ");
double numero1 = Double.parseDouble(s.nextLine());
System.out.print("Introduzca el segundo número: ");
double numero2 = Double.parseDouble(s.nextLine());
System.out.println("La media es " + (numero1 + numero2) / 2);
} catch (Exception e) {
System.out.println("Excepción: " + e.getClass());
System.out.println("Error: " + e.getMessage());
System.out.println("No se puede calcular la media. ");
System.out.println("Los datos introducidos no son correctos.");
}
}
}

```

25.1 Control de varios tipos de excepciones

La clase Exception hace referencia a una excepción genérica. Existen muchas subclases Exception de ella como DataFormatException, IOException, IndexOutOfBoundsException, etc.

```

public class ExcepcionesEjemplo3 {

    public static void main(String[] args) {

        int[] array = {1, 2, 3};

        try {
            // Intentamos acceder a un elemento fuera del tamaño del array
            int valor = array[3];
            System.out.println("El valor es: " + valor);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: índice fuera de rango.");
        }
    }
}

```

```

try {
    // Intentamos convertir una cadena no numérica en un entero
    String cadena = "hola";
    int valor = Integer.parseInt(cadena);
    System.out.println("El valor es: " + valor);
} catch (NumberFormatException e) {
    System.out.println("Error: la cadena no es un número.");
}

try {
    // Intentamos dividir entre cero
    int a = 10;
    int b = 0;
    int resultado = a / b;
    System.out.println("El resultado es: " + resultado);
} catch (ArithmeticException e) {
    System.out.println("Error: división entre cero.");
}

}

```

25.2 Lanzamiento de excepciones con throw

La orden `throw` permite lanzar de forma explícita una excepción. Por ejemplo, la sentencia `throw new ArithmeticException()` crea de forma artificial una excepción igual que si existiera una línea como `System.out.println(1 / 0);`.

```

public class ExcepcionesEjemplo4 {

    public static void main(String[] args) {
        try {
            // Llamamos al método dividir
            int resultado = dividir(10, 0);
            System.out.println("El resultado es: " + resultado);
        } catch (ArithmeticException e) {
            System.out.println("Error: división entre cero.");
        }
    }

    public static int dividir(int a, int b) throws ArithmeticException {
        if (b == 0) {
            throw new ArithmeticException();
        }
        return a / b;
    }
}

```

25.3 Declaración de un método con throws

Hemos visto en el apartado anterior cómo lanzar una excepción desde una función/método con `throw`. Es muy recomendable indicar de forma explícita en la cabecera que existe esa posibilidad, es decir, que el método en cuestión puede provocar una excepción. Se trata de una declaración de intenciones, algo parecido al `@Override`. Si el IDE que estemos utilizando detecta que un método tiene `throws` en su cabecera y, sin embargo, no hemos gestionado bien la posibilidad de provocar una excepción, nos avisará con el correspondiente mensaje de error.

```
public static void main(String[] args) {
    int a = 5;
    int b = 0;
    try {
        int resultado = Calculadora.dividir(a, b);
        System.out.println("Resultado: " + resultado);
    } catch (IllegalArgumentException e) {
        System.out.println("Error: " + e.getMessage());
    }
}
```

25.4 Creación de Excepciones propias.

Java nos permite crear excepciones propias, hechas a medida. Para ello, no hay más que utilizar una de las características más importantes de la programación orientada a objetos: la herencia. Crear una nueva excepción será tan sencillo como implementar una subclase de `Exception`.

```
public class ExcepcionesEjemplo5 {

    public static void main(String[] args) {
        try {
            verificarEdad(15);
        } catch (EdadInvalidaException e) {
            System.out.println("Ocurrió una excepción: " + e.getMessage());
        }
    }

    public static void verificarEdad(int edad) throws EdadInvalidaException {
        if (edad < 18) {
            throw new EdadInvalidaException("La edad mínima para realizar esta acción es de 18 años.");
        }
    }
}
```

```

    }
    System.out.println("La edad es válida, se puede realizar la acción.");
}

}

class EdadInvalidaException extends Exception {
    public EdadInvalidaException(String mensaje) {
        super(mensaje);
    }
}

```

Diferencia entre throws throw

`throw` y `throws` son palabras clave en Java que se utilizan en el manejo de excepciones, pero tienen diferentes propósitos y usos.

`throw` se utiliza para lanzar una excepción explícitamente desde un método o bloque de código. Cuando se utiliza `throw`, se crea una instancia de una excepción y se lanza al lugar donde se llama al método o bloque de código que contiene la declaración `throw`.

Por ejemplo:

```

java
public void miMetodo(int x) {
    if (x < 0) {
        throw new IllegalArgumentException("El valor de x no puede ser negativo");
    }
    // resto del código del método
}

```

En este ejemplo, si el valor de `x` es negativo, se lanza una excepción `IllegalArgumentException` utilizando la palabra clave `throw`.

Por otro lado, `throws` se utiliza en la firma del método para declarar que el método puede lanzar una excepción de un tipo específico. Cuando se utiliza `throws`, se indica que el método puede lanzar una excepción, pero no necesariamente la lanzará.

Por ejemplo:

```

java
public void miMetodo(int x) throws IllegalArgumentException {
    if (x < 0) {
        throw new IllegalArgumentException("El valor de x no puede ser negativo");
    }
}

```

```

    }
    // resto del código del método
}

```

En este ejemplo, se utiliza la palabra clave `throws` en la firma del método para indicar que el método `miMetodo` puede lanzar una excepción `IllegalArgumentException`. Esto permite que quien llame al método pueda tomar las medidas necesarias para manejar la excepción.

En resumen, `throw` se utiliza para lanzar una excepción explícitamente desde un método o bloque de código, mientras que `throws` se utiliza en la firma del método para declarar que el método puede lanzar una excepción de un tipo específico.

25.5 Recomendaciones sobre el tratamiento de excepciones.

1º antes de usar excepciones, procurar realizar comprobaciones previas sencillas.

26 POLIMORFISMO

En Java, el polimorfismo se refiere a la capacidad de un objeto de una clase específica para ser tratado como si fuera un objeto de una clase padre más genérica. Esto significa que un objeto de una clase puede tomar muchas formas diferentes y puede ser utilizado de diferentes maneras según el contexto.

Hay dos tipos de polimorfismo en Java: el polimorfismo de tiempo de compilación (también conocido como enlace estático) y el polimorfismo de tiempo de ejecución (también conocido como enlace dinámico).

El polimorfismo de tiempo de ejecución

El polimorfismo de tiempo de ejecución se produce cuando el enlace de un método o una variable se realiza en tiempo de ejecución. Esto significa que la clase y los métodos que se utilizarán se seleccionan en el momento en que se ejecuta el programa.

Aquí hay un ejemplo de polimorfismo en Java:

```

public class Animal {
    public void hacerSonido() {
        System.out.println("El animal hace un sonido");
    }
}

public class Perro extends Animal {
    public void hacerSonido() {
        System.out.println("El perro ladra");
    }
}

public class Gato extends Animal {

```

```

    public void hacerSonido() {
        System.out.println("El gato maulla");
    }
}

public class PolimorfismoMain{
    public static void main(String[] args) {
        Animal animal1 = new Perro();
        Animal animal2 = new Gato();

        animal1.hacerSonido(); // salida: El perro ladra
        animal2.hacerSonido(); // salida: El gato maulla
    }
}

```

En este ejemplo, tenemos una clase `Animal` con un método `hacerSonido()`. También tenemos dos clases hijas, `Perro` y `Gato`, que extienden la clase `Animal` y sobrescriben el método `hacerSonido()` con su propio comportamiento. Luego, en la clase principal `PolimorfismoExample`, creamos dos objetos, `animal1` y `animal2`, que son instancias de las clases `Perro` y `Gato`, respectivamente, pero se declaran como objetos de la clase `Animal`. Cuando llamamos al método `hacerSonido()` en estos objetos, se ejecuta la versión correspondiente de la clase hija, lo que demuestra el polimorfismo en acción.

Polimorfismo en tiempo de compilación

El polimorfismo en tiempo de compilación (también conocido como polimorfismo estático o polimorfismo temprano) se refiere al uso de sobrecarga de funciones o métodos en un lenguaje de programación. Un ejemplo de polimorfismo en tiempo de compilación sería el siguiente:

En este ejemplo, la clase `Calculadora` define tres métodos llamados `sumar`, pero cada uno de ellos tiene diferentes tipos y cantidades de parámetros de entrada. En el método `main`, se llaman a los tres métodos `sumar` con diferentes parámetros, lo que resulta en diferentes resultados.

```

public class Calculadora {
    public static int sumar(int a, int b) {
        return a + b;
    }

    public static int sumar(int a, int b, int c) {
        return a + b + c;
    }
}

```

```

        public static double sumar(double a, double b) {
            return a + b;
        }
    }

    public class CalculadoraMain {
        public static void main(String[] args) {
            int suma1 = Calculadora.sumar(2, 3); // Output: 5
            int suma2 = Calculadora.sumar(2, 3, 4); // Output: 9
            double suma3 = Calculadora.sumar(2.5, 3.5); // Output: 6.0
        }
    }
}

```

27 APENDICE

27.1 Expresiones lambda.

Ejemplo

```
import java.util.HashMap;
```

```

public class HashMapExample {
    public static void main(String[] args) {
        HashMap<String, String> emails = new HashMap<>();

        // agregar correos electrónicos al HashMap
        emails.put("Juan Perez", "juan.perez@example.com");
        emails.put("Maria Rodriguez", "maria.rodriguez@example.com");
        emails.put("Pedro Gomez", "pedro.gomez@example.com");

        // obtener un correo electrónico por su clave
        String email = emails.get("Maria Rodriguez");
        System.out.println("El correo electrónico de Maria Rodriguez es " + email);

        // recorrer todos los elementos del HashMap
        for (String key : emails.keySet()) {
            String value = emails.get(key);

```



```

        System.out.println(key + " -> " + value);
    }
}
}

```

las expresiones lambda son funciones anónimas, es decir, funciones que no necesitan una clase. su sintáxis básica se detalla a continuación:

(parámetros) -> { cuerpo-lambda }

1. El operador lambda (->) separa la declaración de parámetros de la declaración del cuerpo de la función.
2. Parámetros:
 - Cuando se tiene un solo parámetro no es necesario utilizar los paréntesis.
 - Cuando no se tienen parámetros, o cuando se tienen dos o más, es necesario utilizar paréntesis.
3. Cuerpo de lambda:
 - Cuando el cuerpo de la expresión lambda tiene una única línea no es necesario utilizar las llaves y no necesitan especificar la clausula return en el caso de que deban devolver valores.
 - Cuando el cuerpo de la expresión lambda tiene más de una línea se hace necesario utilizar las llaves y es necesario incluir la clausula return en el caso de que la función deba devolver un valor .

Ejemplos:

Algunos ejemplos de expresiones lambda pueden ser:

- `z -> z + 2`
- `() -> System.out.println(» Mensaje 1 «)`
- `(int longitud, int altura) -> { return altura * longitud; }`

- (String x) -> {
 String retorno = x;

Ejemplo

```
switch (mes) {  
    case "Enero", "enero", "1" -> {  
        System.out.print("Introduzca su día de nacimiento: ");  
        dia = s.nextInt();  
        if (dia > 0 && dia < 20) {  
            horoscopo = "Capricornio";  
        } else if (dia >= 20 && dia <= 31) {  
            horoscopo = "Acuario";  
        } else {  
            horoscopo = "Jaja, te crees gracioso?";  
        }  
    }  
}
```

Ejemplo con colecciones

Aquí te dejo un ejemplo de una función lambda en Java:

```
java  
import java.util.ArrayList;  
import java.util.List;  
  
public class LambdaEjemplo {  
    public static void main(String[] args) {  
        List<Integer> numeros = new ArrayList<Integer>();  
        numeros.add(1);  
        numeros.add(2);  
        numeros.add(3);  
        numeros.add(4);  
        numeros.add(5);  
  
        // Función lambda que filtra los números pares  
        List<Integer> numerosPares = numeros.stream()  
                                            .filter(n -> n % 2 == 0)  
                                            .collect(Collectors.toList());  
  
        // Imprime la lista de números pares  
        System.out.println("Números pares: " + numerosPares);  
    }  
}
```

En este ejemplo, se utiliza una función lambda para filtrar los números pares de una lista de enteros utilizando la API Stream de Java 8. La función lambda se define en la expresión `n -> n`

`% 2 == 0`, que es una expresión lambda que toma un entero `n` como parámetro y devuelve `true` si `n` es divisible por 2 (es decir, si es par).

La función lambda se utiliza en el método `filter` de la API Stream para filtrar los números de la lista que cumplen con la condición especificada. La función `collect` se utiliza para recolectar los elementos filtrados en una nueva lista.

Finalmente, se imprime la lista de números pares utilizando el método `println` de la clase `System.out`.

Recuerda

En Java, una función lambda es una forma concisa de expresar una función anónima. Una función lambda se define como una expresión que se puede pasar como un parámetro a otro método o función. Las funciones lambda se utilizan para escribir código más legible y conciso, especialmente cuando se trabaja con colecciones o interfaces funcionales.

En términos técnicos, una función lambda se compone de tres partes principales: los parámetros, la flecha lambda y el cuerpo de la función. La sintaxis general de una función lambda en Java es la siguiente:

```
rust
(parametros) -> { cuerpo de la función }
```

Los parámetros son una lista separada por comas de los argumentos que la función lambda aceptará. La flecha lambda `->` separa los parámetros del cuerpo de la función. El cuerpo de la función puede ser una expresión simple o un bloque de código que realiza una tarea determinada.

Las funciones lambda se pueden utilizar para implementar interfaces funcionales en Java, que son interfaces que tienen un solo método abstracto. En lugar de escribir una clase que implemente la interfaz funcional, se puede crear una función lambda que implemente el método abstracto de la interfaz.

Las funciones lambda se introdujeron en Java 8 como parte del soporte de programación funcional en el lenguaje. Desde entonces, las funciones lambda se han convertido en una característica popular de Java y se utilizan ampliamente en aplicaciones modernas.

27.2 JOptionPane.

`JOptionPane` es una clase que nos provee una conjunto de ventanas de dialogo que es ideal, para mostrar mensajes al usuario. Ya sean informativos, advertencias, errores, confirmaciones... O incluso tenemos la posibilidad de solicitar la introducción de un dato.

- **`OptionPane.showMessageDialog()`** nos permite mostrar un mensaje.
- **`JOptionPane.showInputDialog()`** nos permite la entrada de datos (similar al famoso Scanner de la consola).

- **JOptionPane.ShowDialog()** nos permite hacer preguntas con varias confirmaciones. Por ejemplo: Sí, No, Cancelar.
- **JOptionPane.showMessageDialog()** engloba/unifica los 3 anteriores diálogos.

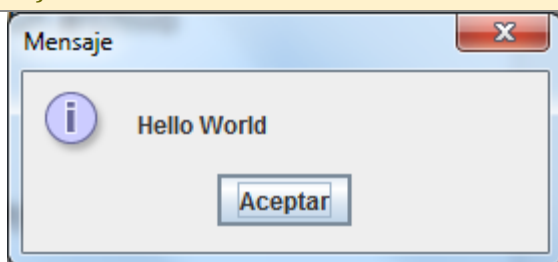
Tipos de iconos de un JOptionPane

Icon	Code	IDE Value
No icon	JOptionPane.PLAIN_MESSAGE	-1
	JOptionPane.ERROR_MESSAGE	0
	JOptionPane.INFORMATION_MESSAGE	1
	JOptionPane.WARNING_MESSAGE	2
	JOptionPane.QUESTION_MESSAGE	3

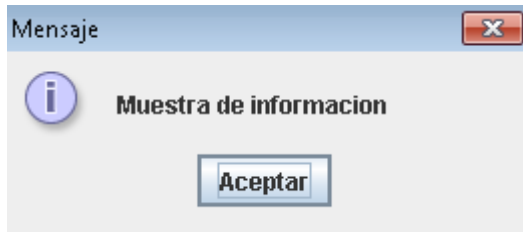
27.2.1 showMessageDialog

```
import javax.swing.JOptionPane;

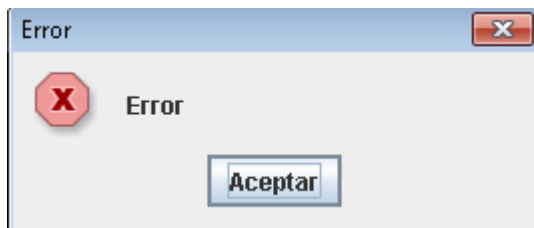
public class HelloWorld {
    public static void main(String[] args) {
        JOptionPane.showMessageDialog(null, "Hello World");
    }
}
```



- **showMessage:** nos permite mostrar información, es como **System.out.println**. Alguno de sus métodos sobrecritos son:
 - **showMessage(parentComponent, message):** es la forma simple de mostrar un mensaje. Por ejemplo, **JOptionPane.showMessageDialog(null, «Muestra de informacion»);**

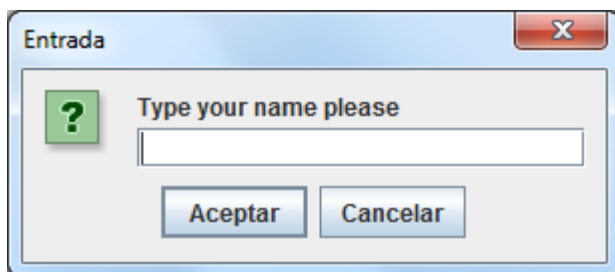


- **showMessageDialog(parentComponent, message, title, messageType):** nos permite personalizar aun más la muestra de información, como su titulo y el icono. Por ejemplo, `JOptionPane.showMessageDialog(null, «Error», «Error», JOptionPane.ERROR_MESSAGE);`



Las constantes de información y advertencia, son iguales que el anterior método.

27.2.2 Método *JOptionPane.showInputDialog()*



El siguiente ejemplo muestra cómo podemos usar este Dialog.

```
import javax.swing.JOptionPane;

public class ShowInputDialogExample {
    public static void main(String[] args) {
        String name = JOptionPane.showInputDialog("Type your name please");
        JOptionPane.showMessageDialog(null, "Hello " + name);
    }
}
```

showInputDialog: nos permite introducir información que usaremos en la aplicación, este método devuelve un String con lo que hayamos escrito. Alguno de sus métodos sobrescritos son:

- **showInputDialog(Object message):** permite mostrar un simple mensaje al dialogo, este sera una cadena.
- **showInputDialog(parentComponent, message, title, messageType):** nos permite personalizar aun más el dialogo, con un titulo y un icono (error, información, advertencia, pregunta o plano). Por ejemplo, **JOptionPane.showInputDialog(null, «Introduce un dato», «Titulo», JOptionPane.INFORMATION_MESSAGE);**



También se puede cambiar el icono a error (**JOptionPane.ERROR_MESSAGE**), advertencia (**JOptionPane.WARNING_MESSAGE**), pregunta (**JOptionPane.QUESTION_MESSAGE**) o plano (**JOptionPane.PLAIN_MESSAGE**).



En este ejemplo hemos configurado que al pulsar aceptar haga una cosa u otra.

```
import javax.swing.JOptionPane;

public class ShowInputDialogExample {
    public static void main(String[] args) {
        String name = JOptionPane.showInputDialog("Type your name please");

        if (name==null){
            JOptionPane.showMessageDialog(null, "Ha cancelado, gracias ");
        }else{
            JOptionPane.showMessageDialog(null, "Ha pulsado aceptar, su nombre es " +
name);}
    }
}
```

27.2.3 Método *JOptionPane.showConfirmDialog()*

showConfirmDialog: nos permite elegir una opción. Devuelve un código numérico que están definidos como constantes. Algunos métodos sobrescritos son:

- **showConfirmDialog(parentComponent, message):** muestra un mensaje con las opciones **Si, No y Cancelar**, cada una de las opciones están definidas con una constantes (nos ayuda a indicar la opción sin saber que numero devuelve), recuerda que debes invocar las constantes con `JOptionPane`:
 - **YES_OPTION:** lo devuelve cuando pulsamos en si.
 - **NO_OPTION:** lo devuelve cuando pulsamos en no.
 - **CANCEL_OPTION:** lo devuelve cuando pulsamos en cancelar.
 - **OK_OPTION:** lo devuelve cuando pulsamos en aceptar.

showConfirmDialog(parentComponent, message, title, optionType): nos permite personalizar el dialogo, indicándole un titulo y su tipo de opción (definido como constante). Las constantes son:

- **OK_CANCEL_OPTION:** muestra la opción aceptar y cancelar.
- **YES_NO_CANCEL_OPTION:** muestra la opción si, no y cancelar.
- **YES_NO_OPTION:** muestra la opción si y no.
- **showConfirmDialog(parentComponent, message, title, optionType, messageType):** es igual que el anterior pero indicándole el tipo de mensaje (advertencia, error o información), se definen igual que en el primer método que hemos visto.

Veamos algún ejemplo de este ultimo método:

```
1int codigo=JOptionPane.showConfirmDialog(null, "¿Quieres un euro para una buena causa?", "Donacion",
2    if (codigo==JOptionPane.YES_OPTION){
3        System.out.println("Has pulsado en SI");
4    }else if(codigo==JOptionPane.NO_OPTION){
5        System.out.println("Has pulsado en NO");
6    }
```

27.2.4 Método *JOptionPane.showOptionDialog()*

```
import javax.swing.JOptionPane;
public class ShowOptionDialogExample {

    public static void main(String[] args) {
        /* Simple JOptionPane ShowOptionDialogJava example */
        String[] options = { "rock", "paper", "scissors" };
        var selection = JOptionPane.showOptionDialog(null, "Select one:", "Let's play a game!",
                                                    0, 3, null, options, options[0]);

        if (selection == 0) {
            JOptionPane.showMessageDialog(null, "You chose rock!");
        }
        if (selection == 1) {
            JOptionPane.showMessageDialog(null, "You chose paper.");
        }
        if (selection == 2) {
            JOptionPane.showMessageDialog(null, "You chose scissors!");
        }
    }
}
```

